

Bachelor's Thesis

For the degree programme Business and Engineering

Back-End Development for Project Nilbye's Prototype Device

In cooperation with
Werk:Raum

Author: Horacio Serrano Leon

Matriculation no.: 3821045

Submission date: 22.04.2025

1st Examiner: Andreas Schiffler

2nd Examiner: Bastian Engelmann

Contents

List of Figures	I
List of Tables.....	IV
List of Equations	V
List of Abbreviations.....	VI
Introduction	1
1 Project Nilbye	2
2 Theoretical Framework	5
3 Prototype Description	18
4 Training.....	27
5 Software Development	35
6 Results	49
7 Results Analysis	60
8 Recommendations	63
Conclusion	66
Bibliography	67
Appendix	72

List of Figures

Figure 1. Werk:Raum association with the THWS and Logo.....	2
Figure 2. Project Nilbye's Concept Presentation.....	2
Figure 3. Local villager using tin drum as deterrence method.....	3
Figure 4. Ineffective fences found near the crop sites.....	3
Figure 5. Machine Learning Main Approaches.....	6
Figure 6. Machine Learning vs Deep Learning comparison.....	7
Figure 7. Deep Learning Workflow.....	8
Figure 8. Convolutional Neural Network layers..	9
Figure 9. Specific and Generic Objects.....	10
Figure 10. Classification and Segmentation..	11
Figure 11. Classification with single and multiple objects.	11
Figure 12. R-CNN Regions.....	12
Figure 13. SSD Architecture.....	13
Figure 14. CNN Architecture.....	13
Figure 15. MultiSound acoustic bird and pigeon deterrent.....	14
Figure 16. Seiche Acoustic Deterrent Device.	14
Figure 17. Prototype Device Network Concept.....	18
Figure 18. NVIDIA Jetson Nano Orin.....	19
Figure 19. Dahua PTZ Camera.	20
Figure 20. Visaton FR 8 WP Loudspeaker.....	20
Figure 21. D-TECH LED Round Blue Light.	20
Figure 22. Kemo Ultrasonic High-Power Generator.....	20
Figure 23. HC-SR04 Ultrasonic Sensor.....	21
Figure 24. HC-SR501 PIR (Passive Infrared Sensor).....	21
Figure 25. Enjoy Solar Panel.....	21
Figure 26. Litionite Power Bank.	21
Figure 27. Basic YOLO functioning.....	21
Figure 28. Non-Maximum Suppression removal of overlapping bounding boxes.....	23
Figure 29. Evolution of YOLO.....	23
Figure 30. YOLO versions comparison with mAP vs FPS.	24
Figure 31. Detection and Deterrence System Layer.....	26
Figure 32. India's Research Team examples of inadequate images	28

Figure 33. India's Team research image samples.	29
Figure 34. Internet image samples.....	29
Figure 35. Example of labeling with Labelme.....	29
Figure 36. Dataset Overall Structure.....	30
Figure 37. Train Structure.....	30
Figure 38. Validation Structure.	30
Figure 39. Data.yaml file.....	31
Figure 40. Desktop use for the Training Process.....	32
Figure 41. Model I Base Code Variant Flowchart.	37
Figure 42. Model II Base Code Variant Flowchart.	38
Figure 43. USB EMEET camera.	39
Figure 44. Setup for rapid detection test.	39
Figure 45. Model I Enhanced Code Variant Flowchart.....	41
Figure 46. Model II Enhanced Code Variant Flowchart.....	42
Figure 47. Arduino Mega Elegoo 2560 R3.....	43
Figure 48. Blue LED.....	43
Figure 49. Passive Buzzer.	43
Figure 50. 220 Ohm Resistance.	43
Figure 51. USB Cable.	44
Figure 52. Bretboard.	44
Figure 53. Jumper Cables M-F & F-M.....	44
Figure 54. Demonstration Device networking.	45
Figure 55. Arduino board connection diagram	46
Figure 56. Setup for advanced detection test.....	48
Figure 57. Dahua ConfigTool.	48
Figure 58. YOLOv5 Training Results.....	50
Figure 59. YOLOv5 Confusion Matrix.	51
Figure 60. YOLOv8 Training Metrics.....	51
Figure 61. YOLOv8 Confusion Matrix.	52
Figure 62. Examples of correct classifications with partial and complete false positives.....	53
Figure 63. Examples of correct classifications with partial and complete false positives.....	54
Figure 64. YOLOv5 YOLOv8 Detection Comparison Chart..	54
Figure 65. YOLOv5 vs YOLOv8 High Confidence Comparison Chart.....	55
Figure 66. Examples of correct classification, false negatives, and false positives.....	56

Figure 67. Examples of correct classification, false negatives, and false positives.....	57
Figure 68.YOLOv5 YOLOv8 Detection Comparison Chart.	57
Figure 69. YOLOv5 vs YOLOv8 High Confidence Comparison Chart.....	58
Figure 70. Examples of correct identification.	59
Figure 71. Examples of correct identification using the EMEET USB Camera.....	59
Figure 72. Project Nilbye’s Detection Gallery.....	65
Figure 73. Model I Base Code Variant.....	72
Figure 74. Model II Base Code Variant.....	73
Figure 75. Arduino Code.....	74
Figure 76. Model I Enhanced Code Variant.....	75
Figure 77. Model II Enhanced Code Variant.....	76

List of Tables

Table 1. Low to mid-frequency ADDs.....	15
Table 2. Ultrasonic ADDs.....	15
Table 3. YOLOv5 vs. YOLOv8 Training Metrics Comparison.....	52

List of Equations

Equation 1. Logic Sigmoid Function	9
Equation 1. Logic Sigmoid Function.	9
Equation 2. Basic Neuron Model.....	9
Equation 3. Neuron Output Formula.	10
Equation 4. Intersection over Union Formula.....	49
Equation 5. Precision Formula.	49
Equation 6. Recall Formula. Adapted from [51],[89].	49
Equation 7. F1-Score Formula. Adapted from [89].	50

List of Abbreviations

ADD	<i>Acoustic Deterrence Device</i>
AI	<i>Artificial Intelligence</i>
ANN	<i>Artificial Neural Network</i>
API	<i>Application Programming Interface</i>
CLAHE	<i>Contrast Limited Adaptive Histogram Equalization</i>
CLI	<i>Command Line Interface</i>
CNN	<i>Convolutional Neural Network</i>
CPU	<i>Central Processing Unit</i>
CUDA	<i>Compute Unified Device Architecture</i>
DIN	<i>Deutsches Institut für Normung</i>
DL	<i>Deep Learning</i>
FPS	<i>Frames per Second</i>
GAN	<i>Generative Adversarial Network</i>
GIL	<i>Global Interpreter Lock</i>
GPU	<i>Graphics Processing Unit</i>
GPIO	<i>General-Purpose Input/Output</i>
HDMI	<i>High-Definition Multimedia Interface</i>
IoT	<i>Internet of Things</i>
IoU	<i>Intersection over Union</i>
JIT	<i>Just in Time</i>
LED	<i>Light Emitting Diode</i>
mAP	<i>mean Average Precision</i>
ML	<i>Machine Learning</i>
MQTT	<i>Message Queuing Telemetry Transport</i>
PIR	<i>Passive Infrared Sensor</i>
PTZ	<i>Pan Tilt Zoom</i>
R-CNN	<i>Region-based Convolutional Neural Network</i>
RPN	<i>Region Proposal Network</i>
RTSP	<i>Real-Time Streaming Protocol</i>
SDD	<i>Single Shot Multibox Detectors</i>
SVM	<i>Support Vector Machine</i>
THWS	<i>Technische Hochschule Würzburg-Schweinfurt</i>
TOPS	<i>Tera Operations Per Second</i>
URL	<i>Uniform Resource Locator</i>
USB	<i>Universal Serial Bus</i>
UTF	<i>Unicode Transformation Format</i>
WHH	<i>Welthungerhilfe</i>
YOLO	<i>You Only Look Once</i>

Introduction

The conflicts between small-scale farmers and wildlife often cause significant losses on both sides. These confrontations not only result in damage to livestock, crops but also disrupt local ecosystems and threaten biodiversity. The consequences of this phenomenon become ever present as this conflict between humans and wildlife escalates, due to further human settlement into natural habitats, which highlights the need for sustainable and effective solutions that can benefit both sides.

It is from these necessity and growing challenges that a pioneering partnership between WHH (Welthungerhilfe) and the THWS (Technische Hochschule Würzburg-Schweinfurt) very own start-up Werk: Raum has created the Project Nilbye as an initiative to protect small crops using methods that avoid harming wildlife.

This initiative aims to leverage affordable and practical Real-Time Object Detection combined with Deterrence Systems enhanced by cutting-edge Artificial Intelligence (AI) technology, to empower the local communities in remote areas. The implementation of ADDs (Acoustic Deterrent Devices) that rely on the use of deep learning architectures for image recognition comes as a response to the current ineffective methods used by these small communities, which are hazardous to the local biodiversity, especially to the diverse wildlife found in this environment. These methods aim to leverage advanced and modern technologies related to machine learning to provide a non-lethal solution that can benefit both parties while being sustainable.

To comply with the device's main features of being low-cost, compact, and user-friendly, an initial prototype with a network of small and affordable components is currently in the making that requires back-end development that matches these parameters. This means simple programming, which can handle complex libraries and advanced deep learning models in a short and compact code, that consolidates all the essential functions and is also capable of handling vast amounts of data while being comprehensive enough for any user with the proper technical background to modify, customize, or enhance depending on any changes related to the data or the device's networking.

This Bachelor Thesis will focus primarily on the development of the programming code and subsequent testing, with its proper documentation written for the Detection and Deterrence System. Project Nilbye itself will also be introduced and referenced in this work, along with the Prototype Device being developed. The aim of documenting the programming process would be the creation of a pipeline that can be followed to further improve and develop the source code produced in this study.

1 Project Nilbye

Before understating Project Nilbye's core design, it is necessary to understand what is the current situation in which many small-scale farmers found themselves and how the two organizations that have started this social business project (THWS's Werk:Raum and WHH) aim to tackle the issues for later implementing the necessary technologies to implement wildlife movement prediction for farm protection and address the issues impacting the communities affected by this conflict.



Figure 1. Werk:Raum association with the THWS and Logo. Adapted from [1].



Figure 2. Project Nilbye's Concept Presentation. Adapted from [2].

Welthungerhilfe (WHH) is one of the largest private aid agencies in Germany [3]. The organization has the goal of "Zero Hunger by 2030," and since its foundation in 1962, it has conducted over 11,498 overseas projects in 70 countries [2],[3]. WHH tackles hunger through sustainable, innovative solutions rooted in addressing poverty and climate change impacts [2]. Emphasizing efficient resource utilization and local empowerment, it aims for long-term economic development. Social businesses are key contributors, providing services, generating employment, and aspiring to be self-sustaining entities. Welthungerhilfe invests strategically, seeking to improve prospects for current and future generations by establishing profitable and sustainable ventures that reinvest profits in their mission [2]. Project Nilbye's research draws primary data from existing statistics from rural parts of India, where the human-wildlife conflict has reached its zenith, with a particular case based on the most remote parts of the state of Bihar being used as a prime example to illustrate the severity of the current situation. This is the case for 16-year-old Rekha and her family of seven members [4]. Together they owned 0,5 acres of farmland where the crops consisted of rice and vegetables at a subsistence level [4]. One of the significant challenges Rekha faces is the constant threat posed by wild animals like Nilgai, wild boar, and monkeys to their crops [4].

The aforementioned research also includes other similar cases in different locations around the area that shared many of the underlying issues regarding deterrence methods and crop damage, as well as harm to human and animal life [5]. The research was conducted between the 12th and 13th of October 2023 in the towns of Maran and Pirprahi in Bihar, India, where the locals mainly practice subsistence farming [5]. Both communities have a population size of between 1,000 and 1,200 inhabitants, with migration patterns influenced by the harvest season [5]. The crops are typically half an acre in size,

focusing on rice, wheat, and various vegetables. The threat of wildlife is constantly present, with Nilgais and Wild boars being the animal species that pose the biggest threat to both crops and structures, forcing the local population to rely on deterrence methods such as bamboo sticks, tin drums, and other drumming utensils, as well as fire torches [5]. Figure 3 shows an example of a local villager using one of these methods. Many of these methods have various degrees of success, with torches being the least effective due to their potential to produce fires in the field. Other traditional methods include fences, which have proven to be ineffective, especially against nilgais, who can easily jump these rudimentary defences. The ineffectiveness of the current fencing, as seen in Figure 4, has prompted the need for innovative and wildlife-specific deterrent methods to protect the crops [5]. As a result of these shortcomings, locals must be constantly involved in implementing these deterrence methods, further impacting their daily lives [5]. This is especially concerning regarding the youngest members of these communities, who are unfortunately frequently absent from school, hindering their progress and development [5].

As part of this research, farmers were interviewed to get a greater insight into how the devices could be better suited for their needs.



Figure 3. Local villager using tin drum as deterrence method. Adapted from [5].



Figure 4. Ineffective fences found near the crop sites. Adapted from [5].

The interviewed subjects were Gita and Priya, both aged 15 [5].

Gita lives in a household of 7 people. Her brother and father must migrate to Punjab for seasonal work. The household leases a farm of 1 acre, where rice is the main crop. Gita needs to invest between 4 to 14 hours a day to scare wildlife using drumming plates [5]. She and her family are willing to adapt new technologies to address the present issues of wildlife intruding on their crops, to the point of investing in the implementation of wildlife deterrents. The current noise levels do not discourage them from utilizing these methods [5].

Priya also lives in a large household of 9 people with relatives who also migrate to Punjab for seasonal jobs. The farm she and her family lease are smaller, with only 0.25 of an acre [5]. Priya has a more gruelling schedule from 8 pm to 2 am to scare wildlife using tins and sticks. Like Gita's, Priya's family is

willing to embrace new technologies while bearing the existing financial requirements and accepting the noise levels that come with wildlife deterrents [5]. Given the information gathered by the research in India, it is possible to conceptualize a system that can identify the species that pose a significant threat to the local communities and their crops. This system will be created to offer a tailor-made solution that manages to offer a twofold benefit [4]:

- Reduce noise pollution with the emission of tailor-made frequencies that target the specific wildlife found in the research.
- A long-term and efficient customization that can enhance the effectiveness of the deterrent strategy, which consists of maintaining game and wildlife away from the crops with minimal human intervention.

To achieve these goals, the system consists of two levels:

1. **Animal Identification Software:** This technology uses sophisticated animal identification and detection software that leverages artificial intelligence (AI) and advanced sensors. The development and implementation of devices that rely on advanced technological methods require a significant amount of research and development investment, including data collection, continuous testing, and constant refinement. The upkeep of the software with its regular maintenance and regular updates incurs additional costs to keep up with ever-evolving wildlife pattern behaviors [4].
2. **Cost-Effective Implementation:** While the software is advanced, the hardware, on the other hand, must be as cost-effective as possible. To this end, it is necessary to plan the creation of a device that can rely on affordable components and materials, to keep expenses at a minimum, so that there is not a significant financial burden for small-scale farmers when the technology is implemented. Additionally, an autonomous energy supply should be included in the system by implementing solar energy. This setup ensures a continuous daily operation, which is essential for deployment in remote rural areas with limited access to electricity. [4].

In accordance with the project's goal, the technology, including the device being developed, should offer the following advantages to the farmers [4]:

- Precision and Customization: The specific species should be targeted so that the deterrence solutions can effectively reduce noise pollution and conserve resources.
- Reliability: Use special frequencies to increase the reliability of the technology, the system, and its devices.
- Long-term effectiveness: The technology must adapt and evolve based on present and real-life conditions so it can remain effective over extended period of times.
- Affordability: The small-scale farms must be able to take full advantage of the technology with limited resources. Therefore, the technology must be affordable enough to work with simple hardware.

2 Theoretical Framework

2.1 Deep Learning Architecture

Deep Learning is a distinct subfield of Machine Learning, which itself is a specialized branch within Artificial Intelligence [6].

Machine Learning is a breakaway from traditional programming due to its method of autonomous learning, which uses data and experiences [7]. With the use of models that rely on sample data or complete datasets, where both input sources are used as “training data”, Machine Learning techniques can execute multiple inexplicitly defined tasks in many devices, such as computers, without any set of strict rules to define every single step [8]. The main differences between Machine Learning and Artificial Intelligence are that, while the former specializes in learning how to replicate core aspects of human intelligence in computers or other devices, the latter focuses on carrying out specialized tasks on computers that are trained to execute using data analysis and pattern recognition [9],[10]. This approach makes Machine Learning reliant on simpler datasets in contrast to Artificial Intelligence, which has a broader scope regarding its potential to carry out even more complex tasks, therefore needing bigger amounts of data to function [11].

Machine Learning

To truly comprehend Deep Learning Architecture as a distinct topic, it is crucial to understand its role in Machine Learning systems and how it distinguishes itself from Shallow Learning Architecture. When analysing the main differences between these two core concepts that allow Machine Learning to function, it is imperative to cover the main approaches of Machine Learning:

- *Supervised Learning*: A system is adapted so that, for every given input data, a target output is produced. As part of the tuples (attributes and labels), every input is represented as an attribute, and every output as a label [12].
- *Unsupervised learning*: The data comprises input vectors without a particular target output, to fulfill different objectives such as clustering, density estimation, and visualization [12].
- *Semi-supervised learning*: In this combined approach, unlabelled data is used to learn a feature representation of the input given to later be solve a given task using representation [12].

These approaches are better illustrated in Figure 5.

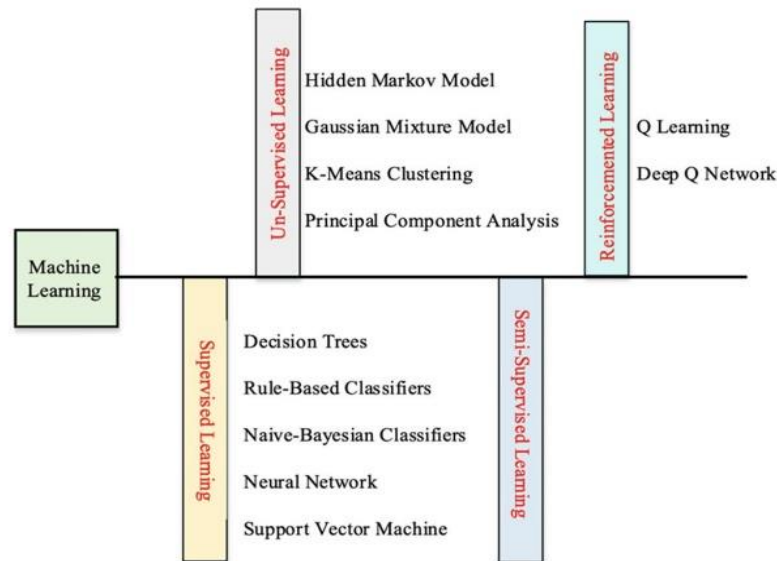


Figure 5. Machine Learning Main Approaches. Adapted from [11].

- *Shallow Learning*: This initial phase, dating back to the mid-20th Century, predates Deep Learning, in the form of algorithms that relied heavily on hand-engineered features and linear models. The plateau of simple yet effective techniques under this stage comprised Linear Regression, Logistic Regression, and Naïve Bayes were the backbone of early Machine Learning applications for simple pattern recognition and decision-making tasks [12],[14]. With the progression of the field, additional algorithms such as Decision Trees, SVM (Support Vector Machine), and KNN (K-Nearest Neighbours) were added to the umbrella of Shallow Learning allowing machines to handle nonlinear relationships and more complex decision boundaries broadening the scope of what machines could do from limited data [12],[14].

Deep Learning

In contrast to Shallow Learning, when it comes to Machine Learning Architectures, Deep Learning is relatively new due to its recent appearance in the 21st century. With multiple hidden layers for learning different features and multiple levels of abstraction. The algorithms used to solve complex problems seek to exploit the unknown structure in the input distribution to discover good representations with higher-level learned features defined in terms of lower-level features. What sets Deep Learning apart from conventional Machine Learning approaches is the possibility of inputting raw data without the extraction of features or definition of a feature vector. The process for learning features involving rules and algorithms from raw data is automated during the training of the models rather than being manual with heavy intervention [12].

In deep learning, a problem is realized in terms of a hierarchy of concepts, with each concept built on top of the others [15]. The difference between Deep Learning and Machine Learning can be seen in Figure 6. The lower layers of the model encode some basic representation of the problem, whereas higher-level layers build upon these lower layers to form more complex concepts. The key aspect of

deep learning is the feature layers are not handcrafted and designed by human engineers; rather, they are learned from data gradually using a general-purpose learning procedure [12],[16].

The process is considered hierarchical because each layer is built upon the previous layer. In this case, the outputs are used as building blocks to construct concepts that vary in complexity depending on the layer's level, where the higher the layer is, the more complex it becomes. The more layers the deep learning model has, the deeper it is, having every layer having an increasingly meaningful representation of the input data. Current deep learning often involves learning tens or even hundreds of successive layers of representation from the training data automatically [12].

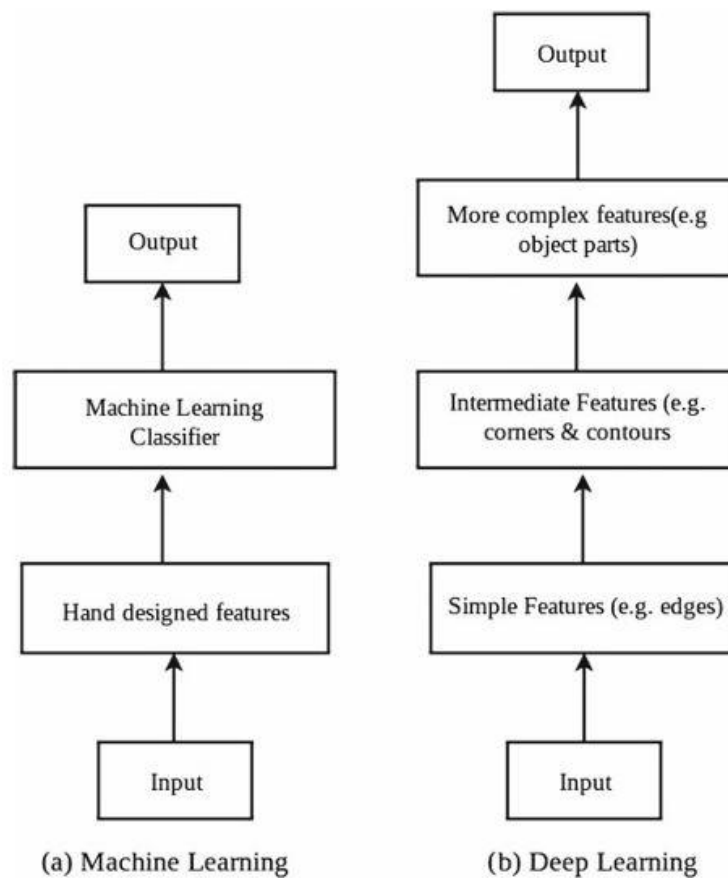


Figure 6. Machine Learning vs Deep Learning comparison. Adapted from [11].

Deep Learning has a very intertwined relationship with neural networks, but it differs from them in very key aspects. Since the advent of ANN (Advanced Neural Networks) in the 1980's the basics of Deep Learning started to develop as its concept because of advancements in this technology. By the early 20th century, the distinctions between the two started to be very clear. Deep Learning, unlike many forms of Neural Networks, including ANN, has more layers, adding levels of complexity and therefore requiring more training time to perform advanced tasks [17]. Equation 2 demonstrates a Basic Neuron Model.

Deep Learning Networks

Deep Learning Architectures are comprised of several hidden layers, commonly referred to as Deep Networks, which learn different features with multiple levels of abstraction. With heavy use of supervised learning, the training process involves the use of large datasets. The working of the networks relies on mapping the input to a given target through a sequence of layered transformations, which themselves learn by exposure to the training examples. These transformations being a result of the layer's application, are determined by a collection of "knobs" like parametrical adjusters called weights, which shape the output-input function. A Deep Learning Network can have millions of weights that regulate hundreds of thousands of parameters [12].

In this context that, the weights play a primordial role in the learning process, given that for correctly mapping the training examples with their associated targets, the weights are gradually adjusted in all layers of the network, being the object of the training itself to find values for the weight that minimize the error rate related to all associations. To achieve this adjustment, during the training is important to define how to close the gap between the output in every iteration and the desired value, this gap being a loss function. When the values of the weights are obtained, a feedback loop is created to decrease the loss score in the function, using a central algorithm in Deep Learning, the backpropagation algorithm, which calculates the target's derivative to the input of one of the layers' modules, backward from the gradient with relation to the output. This algorithm has its forward counterpart, where the outputs are calculated for a set of specific inputs [12].

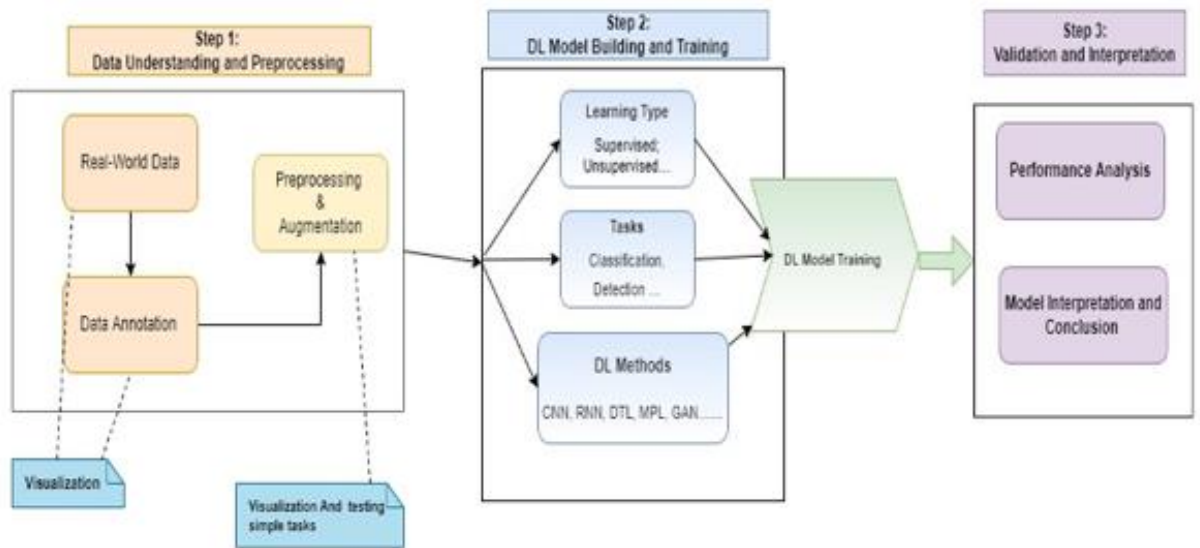


Figure 7. Deep Learning Workflow. Adapted from [18].

As shown in Equation 1, in the Logic Sigmoid Function α denotes activation and is a linear combination of the inputs $x = \{x_1, \dots, x_p\}$ and $w = \{w_1, \dots, w_2\}$ are the weights [18].

$$\sigma(\alpha) = \frac{1}{1 + e^{-\alpha}}$$

Equation 1. Logic Sigmoid Function. Adapted from [17].

$$\hat{y} = \delta(\omega_0 + \sum_{i=1}^P w_i x_i)$$

Equation 2. Basic Neuron Model. Adapted from [17].

ω_0 being the extra weight has nothing to do with the input [18].

Convolutional Neural Networks

With its frequent use in a wide range of applications that rely on Deep Learning for their execution, such as computer vision, voice processing, and face recognition, this algorithm is advantageous due to its low reliance on human input, unlike its predecessors [17].

CNNs take inspiration from the organization of the animal visual cortex, giving a strategy where weight sharing is used for exploiting similar structures found in an image [12],[17]. By focusing on specific weights, the number of parameters required for learning is drastically reduced, making the number of weights necessary independent from the image size [17]. Figure 8 illustrates how CNN layers function.

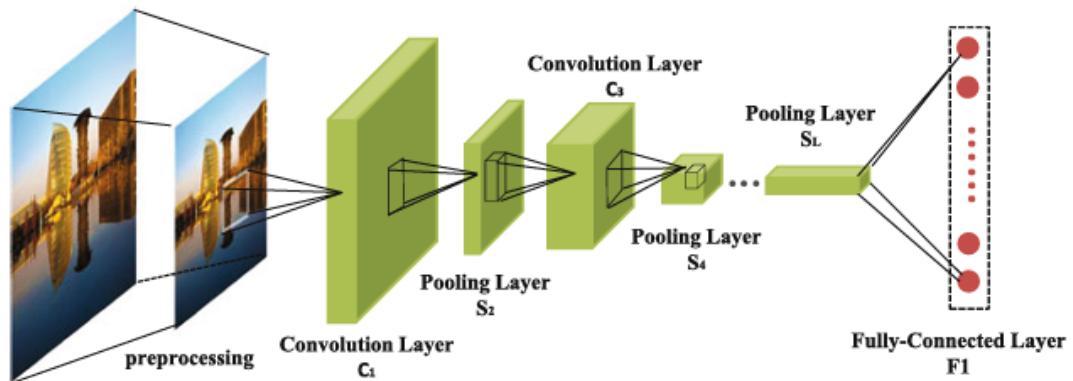


Figure 8. Convolutional Neural Network layers. Adapted from [17].

A common CNN is comprised of many convolutional layers, where the data is convolved with many K learnable kernels $W = \{W_1, W_2, \dots, W_K\}$ plus biases $b = \{b_1, b_2, \dots, b_K\}$. A new feature map comes into play with X_k resulting from the input of the convolution results into an element-wise nonlinear function $\sigma()$ [17]. This output can be seen in Equation 3.

$$x_k^{l+1} = \sigma(w_k^l * x^l + b_k^l)$$

Equation 3. Neuron Output Formula. Adapted from [16].

The process consists of the pooling layer following the convolutional layer to perform a nonlinear down-sampling. Then the feature map is partitioned into nonoverlapping rectangles and outputs maximum values for each sub-region. After several convolutional and pooling layers, the high-level reasoning within the network is implemented with fully connected (FC) [17],[19].

Computer Vision

The areas of this research field include video surveillance, robotics, and human interaction applications, which, due to environmental factors related to lighting and variations in scale and occlusion, present complex challenges for their proper execution. The idea behind computer vision is to instruct machines to acquire knowledge and understand visual content and representation, mainly in the shape of images. Object detection is a subfield of computer vision that, due to the environmental conditions, requires intense use of deep learning networks [20],[21].

Object Detection

As one of the most challenging problems in computer vision, object detection can be defined as the location of object instances from several categories found across large quantities of image data. These categories can include humans, animals, transportation methods, and others. If a matching instance is located, then the goal is to return the spatial location and the extent of the object's instance. Object detection can be grouped into two categories, as seen in Figure 9, detection of specific instances and detection of group categories [22]. As seen in the image, the distinction between these two categories is that the detection of particular instances aims to detect exact objects with a single instance correctly. In contrast, category detection aims to identify objects as members of a general class with several (sometimes unrecognized) instances [22].



Figure 9. Specific and Generic Objects. Adapted from [22].

For both specific and generic object detection, the main goal is to return a matching classification when an instance is detected by returning its spatial location and extent corresponding to the object being detected. This is done using either bounding boxes, which are axis-aligned rectangles, a segmentation mask, or a closed boundary, as shown in Figure 10 [22].

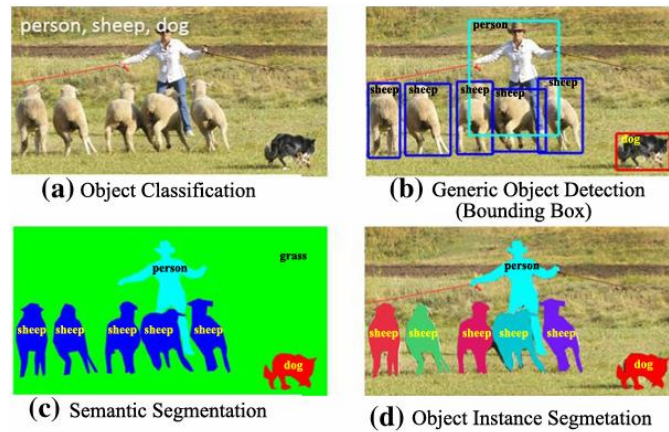


Figure 10. Classification and Segmentation. Adapted from [22].

The bounding boxes tend to be the preferred spatial location representation, but segmentation is closely linked to future developments [22].

A key part of generic object detection is image classification, in which an object within a given image is classified into one predefined category once an instance is detected. Image detection and classification pose many challenges, such as object occupancy, either with a majority of pixels around 70% to 80% or with a minority of around 10% or less, as seen in Figure 11. Other reasons include low image resolution, object size, proper labeling, and overlapping of visual objects [23].

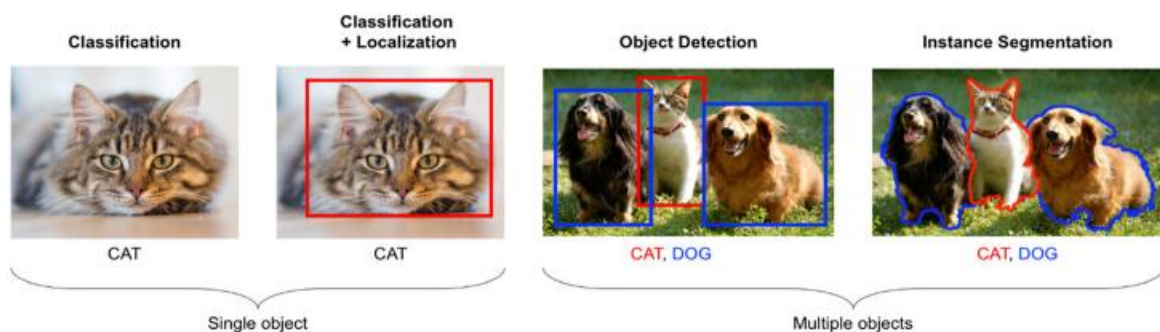


Figure 11. Classification with single and multiple objects. Adapted from [23].

With these concepts defined, the role of CNNs in object detection becomes clearer. The challenges posed for the proper detection and classification of images, whether their spatial location is represented with bounding boxes or segmentation, have been overcome with the use of CNNs due to their ability to learn hierarchical representations from raw data, where lower layers detect edges and

textures, while deeper layers recognize complex structures and shapes. In addition, CNNs can effectively tackle other problems, such as intra-class variation and inter-class similarities, by learning discriminative features that can distinguish objects based on context and structure. These features make CNNs better suited for these tasks than any other deep-learning networks [17].

Two-Stage Detection Methods

These methods generate a candidate object proposal to classify them into specific categories later. In this method, accuracy is prioritized over speed. The prime model of these methods is R-CNN from 2014, as seen in Figure 12, which uses selective search to generate region proposals and runs on each proposal individually [17].

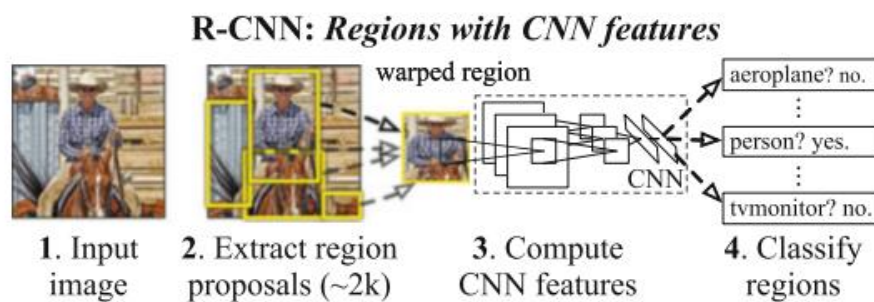


Figure 12. R-CNN Regions. Adapted from [24].

Fast R-CNN was introduced in 2015. In contrast to applying individual CNNs, it extracts features from the entire image first. Then, the region proposals are applied to a shared CNN feature map, making Fast R-CNN slightly faster than the conventional R-CNN but with a still slow region proposal. Finally, Faster R-CNN, which in 2016 managed to replace selective search use in CNNs with a region proposal network or RPN, optimizing time without sacrificing accuracy and improving it significantly from the previously mentioned two-stage methods [17].

One-Stage Detection Methods

Unlike the two methods, the models used in these object detectors extract and classify the object proposals simultaneously, eliminating the region proposal step. They directly predict bounding boxes and class probabilities from an image in a single pass. Speed is prioritized over accuracy in these methods, making them more suitable for real-time applications, such as real-time object detection [17].

The best examples of one-stage methods are the following:

- **SSD or Single Shot Detector (2016):** This is the first major improvement over two-stage models by introducing multi-scale feature maps to detect objects of different sizes and eliminating regional proposal networks (RPNs). However, they used default anchor boxes but suffered from class imbalance (many background detections with few actual objects) [17]. Figure 13 shows the architecture of an SSD.

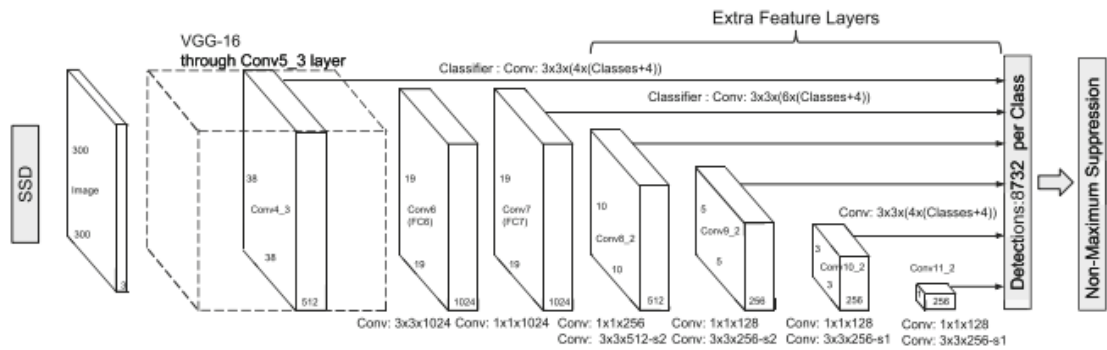


Figure 13. SSD Architecture. Adapted from [25]

- YOLO or You Only Look Once (2016-Present): Eventually overshadowing SSD, YOLO divides the image into a fixed grid and predicts bounding boxes directly from each grid cell. Another key feature is the prediction of bounding boxes from feature maps without extra steps, rather than predefined boxes from feature maps. Speed has considerably improved with this method, making it ideal for real-time object detection. YOLO will be explained in more detail in Section 3 [17]. Figure 14 shows the architecture of a CNN.

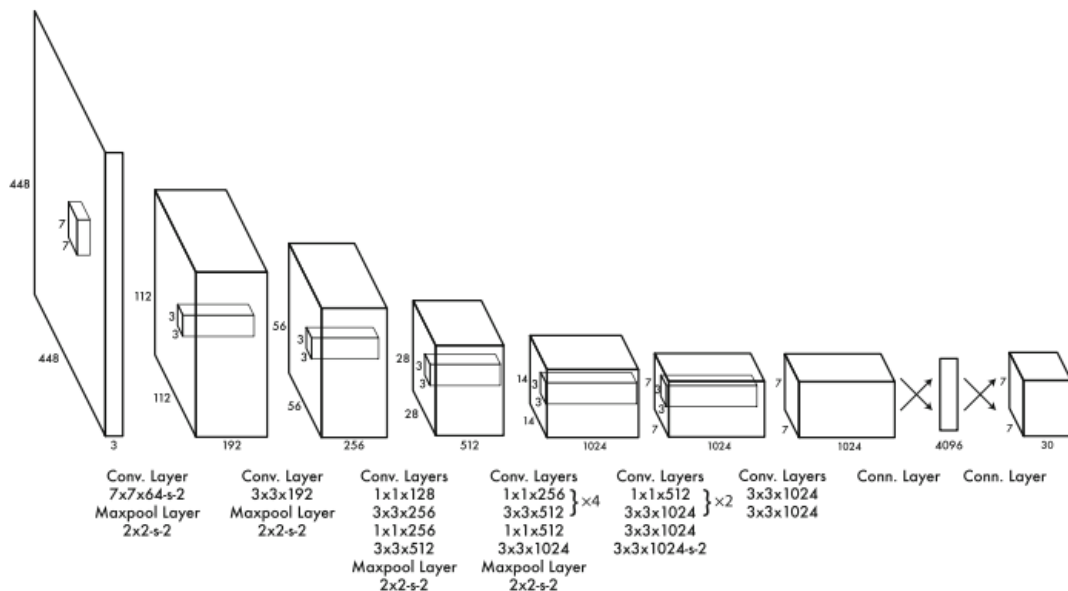


Figure 14. CNN Architecture. Adapted from [26].

2.2 Acoustic Deterrence

The use of sound waves and ultrasonic frequencies to direct wildlife away from a given specific area that can be affected by a collection of species is called acoustic deterrence. The purpose of these practices is to prevent the disruption of wildlife species in human-occupied spaces and different kinds of infrastructures such as fisheries, wind farms, crops, or marine construction sites. In addition, these practices present a non-lethal alternative to the conventional approach that is harmful to humans and wildlife [27],[28].

Acoustic Deterrence can be achieved with the use of:

- Emitting sound that exceeds the animal's auditory threshold, to cause avoidance behaviour.
- Acoustic signals that induce discomfort or disturbance without causing severe pain.
- Using tailor-made ultrasonic frequencies that can be directed to a specific target[27],[29].



Figure 15. MultiSound acoustic bird and pigeon deterrent. Adapted from [30].



Figure 16. Seiche Acoustic Deterrent Device. Adapted from [31].

Acoustic Deterrence Devices (ADDs)

Many devices are created using the principles of acoustic deterrence to be used in many different environments to target a wide variety of animal species. They can vary in both technical complexity and range of applications [27],[29].

ADDs can themselves vary in:

- Frequency range: Ultrasonic or infrasonic frequencies are often used to target specific species with auditory sensitivity.
- Power: An ADD can be low powered below 185 dB re one μ P. When power is high, with higher levels, it is often called an AHD or Acoustic Harassment Device.
- Emission Patterns: ADDs can emit frequencies or sounds either continuously or with random or triggered pulses [28],[32].

Application in Different Habitats

While the most common applications of ADDs are in the deterrence of marine wildlife such as dolphins, pinnipeds (seals, sea lions), and many other similar species, these devices are also used in other habitats where birds and mammals are targeted [29],[33].

Low to Mid-To-Frequency ADDs

Device Type	Frequency Rate	Target Species	Application
<i>Seal Scarers (AHDs - Acoustic Harassment Devices)</i>	<i>5–15 kHz</i>	<i>Seals, sea lions</i>	<i>Fish farms, fisheries</i>
<i>Marine Pingers</i>	<i>1–10 kHz</i>	<i>Dolphins, porpoises, and whales</i>	<i>Bycatch prevention</i>
<i>Bioacoustic Repellents</i>	<i>Species-specific calls (500 Hz–10 kHz)</i>	<i>Birds, wild boars, deer</i>	<i>Agriculture, airfields</i>

Table 1. Low to mid-frequency ADDs. Values adapted from [22],[26].

Ultrasonic ADDs

Device Type	Frequency Rate	Target Species	Application
<i>Ultrasonic Bird Repellents</i>	<i>20–40 kHz</i>	<i>Pigeons, sparrows, crows</i>	<i>Farms, airports, urban areas</i>
<i>Ultrasonic Rodent Repellents</i>	<i>30–60 kHz</i>	<i>Mice, Rats</i>	<i>Storage facilities, homes, and agriculture</i>
<i>Bat Deterrents for Wind Farms</i>	<i>30–50 kHz</i>	<i>Bats</i>	<i>Wind turbines, conservation areas</i>
<i>Wild Boar & Deer Repellents</i>	<i>25–40 kHz</i>	<i>Wild boars, deer</i>	<i>Farmlands, forestry</i>
<i>Marine Ultrasonic Pingers</i>	<i>30–50 kHz</i>	<i>Dolphins, porpoises</i>	<i>Prevent bycatch in fishing nets</i>

Table 2. Ultrasonic ADDs. Values adapted from [22],[26].

2.3 Prototype Device Networking

Prototype Device Networking refers to the design and implementation of data exchange and communication systems between prototype components, ensuring efficient, secure, and low-latency connectivity. This design can vary depending on the system's need, being internet-based with IoT (Internet of Things)-enabled or localized with device-to-device communication (standalone) [34],[35].

IoT Enabled

This type of Prototype Device Networking relies on cloud or internet connectivity and uses wireless communication technologies such as Wi-Fi, ZigBee, and LoRa to connect distributed devices. Application-layer protocols like MQTT are then used to transmit data between devices and cloud services, allowing the network to be monitored and controlled remotely [35],[36].

Standalone

Contrary to the IoT type, this network relies on direct communication between hardware components with the use of wired or low-latency communication methods such as I²C, SPI, UART, PCIe, and Ethernet [37],[38],[39].

2.4 Code Organization

Writing clean and readable programming code that is used in the development of long-term software involves a series of practices and principles that must be followed for efficient maintenance, debugging, updating, and upgrading [40].

Practices

The correct structure of a project directory that enhances code maintainability, collaboration, and scalability is crucial for keeping all software components stored in clearly defined folders.

A well-structured project directory is essential for organizing software components in a way that enhances maintainability, collaboration, and scalability [41].

In this context, source code is a fundamental software component that contains human-readable instructions. These instructions are either interpreted at runtime or compiled into machine-executable code, depending on the programming language used [42].

Execution depends on the programming paradigm. Some languages (e.g., Python, JavaScript) rely on scripts that are executed dynamically, while others (e.g., C++, Java) require compilation before execution [43]. Additional software components, such as data files, model files, configuration settings, and dependency lists, are often necessary for full functionality [44].

Modular Directory Structure

To maintain clarity and separation of concerns, a well-organized project typically follows a modular directory structure. A standard directory layout may include [45]:

- `src/`: Stores source code files and modules.
- `bin/`: Contains compiled binaries or standalone executable files.
- `docs/`: Includes documentation files such as user manuals or API references.
- `tests/`: Houses unit and integration tests.
- `data/`: Stores input datasets, logs, or configuration files.
- `scripts/`: Contains automation scripts, deployment utilities, or preprocessing scripts.
- `requirements.txt` (for Python projects): Lists dependencies required for execution.
- `README.MD`: Provides an overview of the project and basic usage instructions.

By adhering to these principles, developers can create software that is scalable, maintainable, and efficient, ensuring long-term usability and collaboration [45],[46]. Project Nilbye's source code follows the principles outlined in this subsection, and it can be found on the GitHub repository: <https://github.com/hbo-bomb/Project-Nilbye.git>. The repository contains all the scripts and datasets used, with the addition of test results. A complete overview can be found on `README.txt`.

3 Prototype Description

The Prototype Device is powered by a solar cell paired with batteries, connected to the camera and the Nvidia Jetson Nano, which is the central element that controls and powers the loudspeaker, the blue light, and the sensors. The device will be placed in strategic positions inside the crop to effectively cover both the outside and inside areas it seeks to protect from wildlife. Figure 17 shows the Device's Concept.

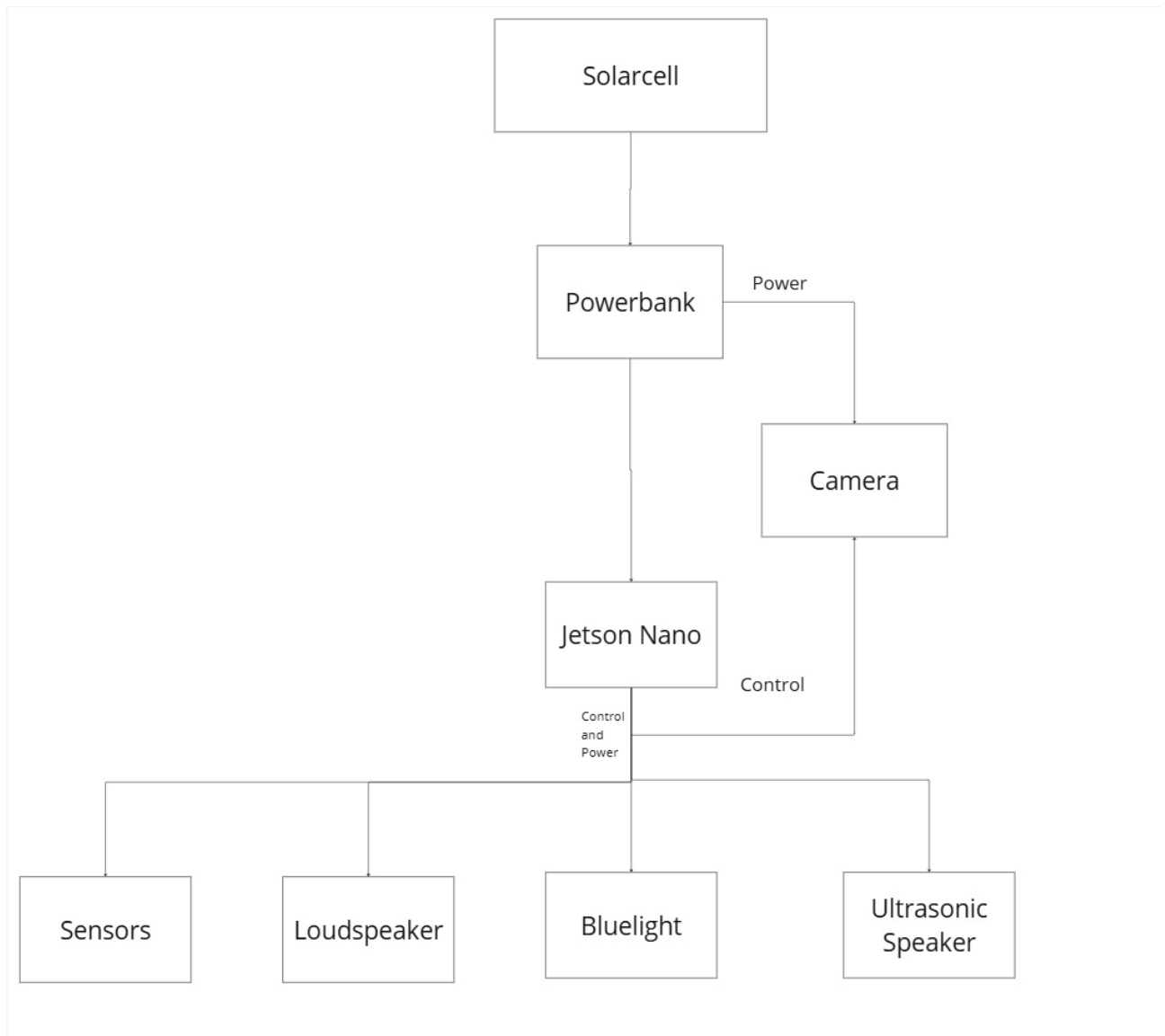


Figure 17. Prototype Device Network Concept.

3.1 Devices and Components

Nvidia Jetson Orin Nano

The NVIDIA Jetson Orin Nano has been chosen as the centrepiece of Project Nilbye's Prototype Device, due to its AI performance within a compact form factor (8GB module delivers up to 67 TOPS) that balances high computational capability and energy efficiency [47]. These features are crucial for Real Time Object detection tasks, especially in systems powered by solar panels. Its configurable power consumption between 7W and 15W allows the device to adjust its energy consumption based on work-load demand, reducing energy waste when the system is idle or operating under lighter loads [47]. In addition, the ability of the Jetson Orin Nano to achieve approximately 10 to 12 frames per second (FPS) when running YOLO models on 1920x1080 resolution aligns with project goals of achieving accurate and precise real-time object detection in a wide variety of conditions[47]. Figure 18 Additional features.

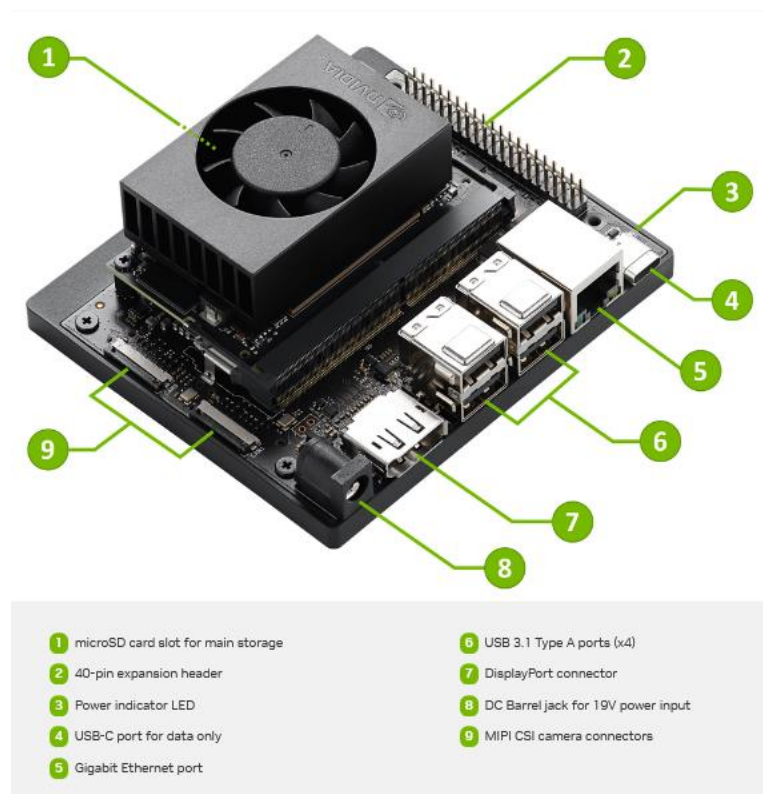


Figure 18. NVIDIA Jetson Nano Orin. Adapted from [37].

NVIDIA Jetson Orin Nano Specifications

According to the Official Jetson Orin Nano Super Developer Kit Datasheet, the specifications are the following [47]:

- Jetson Orin Nano 8GB Module
- GPU: NVIDIA Ampere architecture with 1024 NVIDIA® CUDA® cores and 32 Tensor cores
- CPU: 6-core Arm Cortex-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3
- Memory: 8GB 128-bit LPDDR5 68 GB/s
- Storage: external via microSD slot, external NVMe via M.2 KeyM
- Power: 7W to 15W

The NVIDIA Jetson Orin Nano was configured following the instructions of the Developer Kit Getting Started Guide from NVIDIA's Official Site [48].

Peripheral Components

The Prototype has the following components:

1. Dahua PTZ Camera
2. Visaton FR 8 WP Loudspeaker
3. D-TECH LED Round Blue Light
4. Kemo Ultrasonic High-Power Generator
5. HC-SR04 Ultrasonic-Sensor
6. HC-SR501 PIR (Passive Infrared Sensor)
7. Enjoy Solar Panel
8. Lithium Power Bank



Figure 19. Dahua PTZ Camera.



Figure 20. Visaton FR 8 WP Loudspeaker.



Figure 21. D-TECH LED Round Blue Light.



Figure 22. Kemo Ultrasonic High-Power Generator.



Figure 23. HC-SR04 Ultrasonic Sensor. Adapted from [49].



Figure 24. HC-SR501 PIR (Passive Infrared Sensor). Adapted from [50].



Figure 25. Enjoy Solar Panel.



Figure 26. Litionite Power Bank.

Used Model Architecture YOLO (You Only Look Once)

As mentioned in Section 2, YOLO is a One-State Detection Method that treats object detection as a single-pass regression problem, predicting both bounding boxes and class probabilities in one forward pass of a neural network [51],[52],[53],[54]. Figure 20 illustrates YOLO's basic functioning.

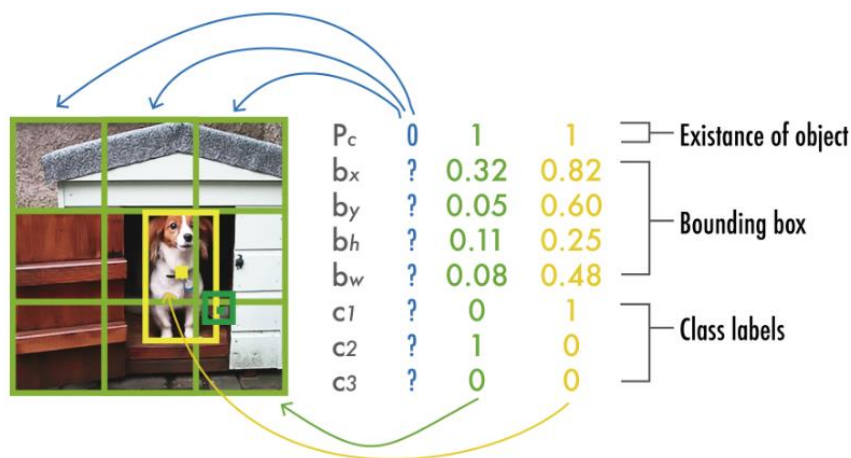


Figure 27. Basic YOLO functioning. Adapted from [53].

The YOLO detection follows these steps [53]:

1. Input Image Processing

- The input image is resized to a fixed size [55].
- The image is divided into an $S \times S$ grid [51],[52],[56].

2. Bounding Box Prediction

- Each grid cell predicts B bounding boxes (x , y , width, height) and a confidence score.

3. Class Probability Prediction

- Each grid cell also predicts C -class probabilities conditioned on the object [57].
- These probabilities and confidence scores are combined to compute final detections.

4. Non-Maximum Suppression (NMS) [58]

- Since multiple bounding boxes may overlap, NMS removes redundant detections, keeping only the best predictions. Figure 28 shows how NMS removes additional bounding boxes.



Figure 28. Non-Maximum Suppression removal of overlapping bounding boxes. Adapted from [53].

YOLO Evolution

- **YOLOv1 (2016):** First introduced the single-stage detection approach [51],[52],[53].
- **YOLOv2 (2017) & YOLOv3 (2018):** Introduced anchor boxes and multi-scale detection to improve accuracy [51],[52],[53].
- **YOLOv4 (2020):** Optimized speed and accuracy with CSPDarknet-53 and advanced data augmentation [52],[53].
- **YOLOv5 (2020):** Implemented in PyTorch, enhancing the ease of training and deployment [53],[54],[59].
- **YOLOv6 (2022):** Industrial-focused optimization with EfficientRep backbone [53], [54], [58],[60].
- **YOLOv7 (2022):** Introduced E-ELAN for better real-time performance [53], [54], [58],[60].
- **YOLOv8 (2023):** Removed anchor boxes (anchor-free design) and improved small object detection [53], [54], [58],[60].

- **YOLOv9 (2024):** Integrated transformer-based feature extraction [61],[62].
- **YOLOv10 (2024):** Hybrid Transformer-CNN model, improving multi-scale detection [61],[62]..
- **YOLOv11 (2025):** Optimized for mobile AI with an efficient Transformer-CNN hybrid model [61],[62].

The complete timeline is shown in Figure 29.

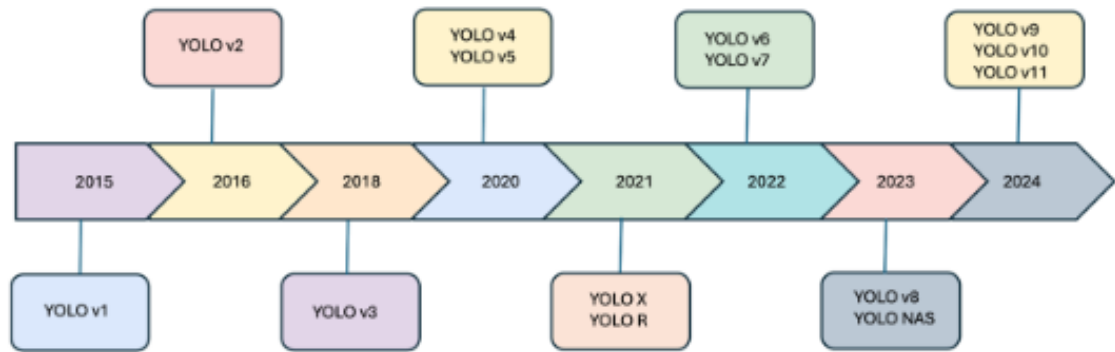


Figure 29. Evolution of YOLO. Adapted from [63].

YOLOv5 and YOLOv8 Implementation

Among the existing YOLO versions (YOLOv1–YOLOv11), YOLOv5 and YOLOv8 were selected for Project Nilbye due to their optimal training and deployment capabilities, hardware compatibility, and speed-accuracy balance, which aligns with the prototype’s real-time object detection needs.

Both versions address key deep learning and object detection challenges, particularly the need for efficient training and fine-tuning on custom datasets, which is crucial given the specific environmental conditions outlined in Project Nilbye [52],[53],[60].

- **YOLOv5** leverages PyTorch, an open-source deep learning framework from Facebook AI (Now Meta) with excellent GPU support and real-time computation capabilities [53],[54].
- **YOLOv8** enhances this with automatic anchor selection and an intuitive API, simplifying deployment and fine-tuning [53],[58],[62].

In terms of speed vs accuracy:

- **YOLOv5** offers minimal latency, making it ideal for fast and simple real-time applications. It achieves 200 FPS on an NVIDIA V100 GPU with a batch size of 32 at 640×640 resolution [53], [59].
- **YOLOv8**, with its anchor-free design and decoupled detection head (separating classification and regression tasks), excels in detecting small and complex objects. It reaches 280 FPS on an NVIDIA A100 GPU using TensorRT [53], [62].

In addition, YOLOv5 and YOLOv8 are part of a YOLO family that is widely used in real-time object detection and is usually paired with newer versions from YOLOv9 onwards, despite being relatively older [53],[58],[62].

The following comparison between the YOLO versions that are most widely used for the tasks present in Project Nilbye, such as real-time object detection, is between accuracy represented by mAP and speed represented by FPS on the MS COCO Dataset from various studies [53],[58],[64].

The COCO dataset (Common Objects in Context) is a large-scale dataset widely used in computer vision for tasks such as object detection, segmentation, and image captioning [65],[66]. It contains over 330,000 images, with over 200,000 labeled, encompassing 1.5 million object instances across 80 object categories. Each image is annotated with object bounding boxes, segmentation masks, and descriptive captions, providing rich contextual information [52],[53].

Performance metrics such as Mean Average Precision (mAP) and Frames per Second (FPS) are usually evaluated on a standard 640x640 pixels image resolution, which provides a balance between detection accuracy and computation efficiency [67]. With higher resolutions, detection accuracy significantly improves, particularly for small objects, but at the cost of reduced inference speed [67],[68],[69].

Both YOLO versions are optimized for embedded AI systems, such as the ones used in the NVIDIA Jetson Orin Nano, unlike newer versions (YOLOv9 & YOLOv10), which lack deployment optimization for edge devices [53],[60],[62]. The accuracy vs. speed trade-off represents a good balance, where computational costs like the ones found in more advanced versions like YOLOv11 are lower [53],[61],[62].

Figure 30 shows the comparison between many YOLO versions considering mAP vs FPS.

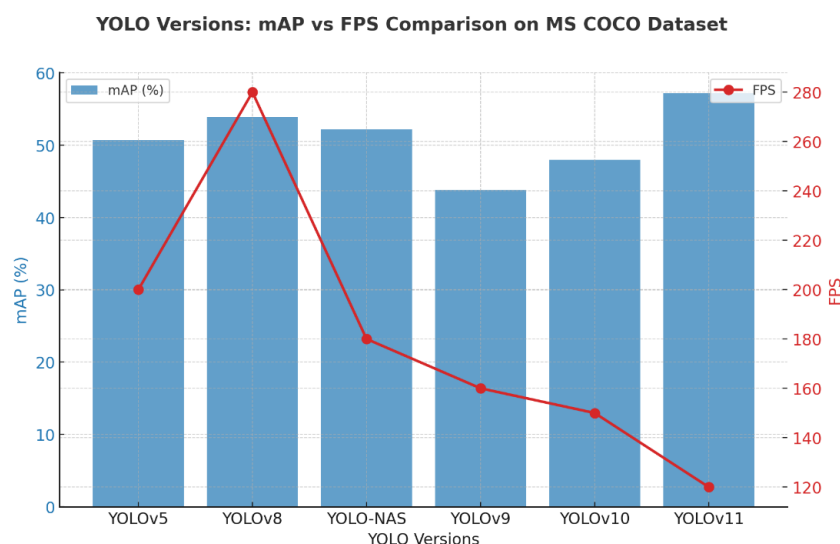


Figure 30. YOLO versions comparison with mAP vs FPS.

3.2 Complete System Overview

During the Prototype Device's functioning, a detection layer is present. The camera mainly activates this overseeing layer, which enables the entire process. It records constantly, and if an object enters its line of sight around 150 m, the camera processes the images that are later sent to the Nvidia Jetson Nano.

After that, the images are compared with the best results of the pre-trained YOLO model. If this comparison returns a positive match with the model's classification that includes many wildlife species, the object is recognized, and the process continues to the first deterrence layer. In this layer, a loudspeaker is activated that plays a recording designed to scare many wildlife species. At the same time, a second deterrence layer is simultaneously activated. Deterrence is carried out in this layer by a blue light meant to emphasize the alert, with the recording redirecting the object away from the area. Both layers last for 60 seconds and get activated. The classified object triggers this process until it leaves the camera's line of sight. This process is not designed to enter a loop but to continuously be activated in a single run. In the case of a classified object approaching the area, a third deterrence layer, overseen by a second detection layer, is in place. The detection layer, in this case, is activated by motion if the classified object enters a specific radius within the area. Motion detection is registered by two sensors that can work independently of each other. The Ultrasonic sensor measures the distance to ensure the classified object is within the specified range, and a PIR (Passive Infrared Sensor) detects motion to confirm the classified object's presence. Either if one or both sensors return a positive result indicating that the object is within range and moving, a speaker designed to play ultrasonic sounds plays a specific frequency tailored for the species. This layer only stops once the classified object has left the range field. Otherwise, this process enters a loop until this condition is fulfilled. The chain of conditions to activate the first detection layer and the first and second deterrence layers is the object entering the camera's line of sight and matching positively with the YOLO model's classification. For the entire system to be active, the classified object must additionally fulfill the condition of being within the range field inside the area after already entering the outside part of the surveillance area as viewed by the camera. The second detection and third deterrence layers are designed to act in case the classified object runs toward the crop rather than away from it. This ensures a more effective and protective method that covers many situations where wildlife is present in the area.

When the object enters the outer boundary, it is detected by the first layer. If the detection returns a positive match from the model's classification, the first and second deterrence layers are activated.

On the next page, Figure 31 illustrates the complete layer workflow, including all the conditions that must be activated.

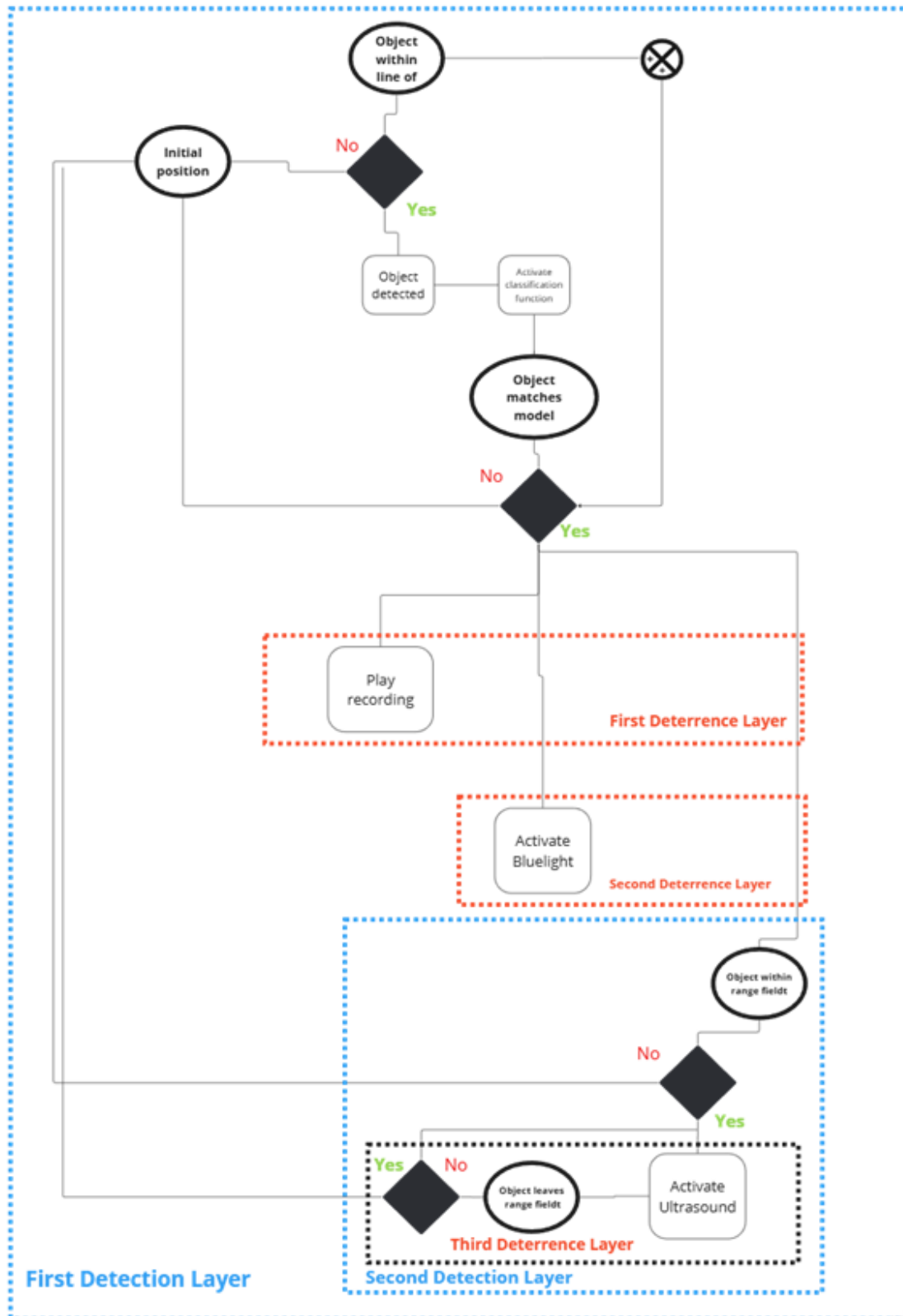


Figure 31. Detection and Deterrence System Layer

4 Training

Project Nilbye's Software is comprised of two essential components. The first is the Weights of two separate Deep Learning Models, created by implementing yolov5s[68] and yolov8s[69] (respectively) as the basis for the training process. Both Model's Weights contain results, one being the best.pt and last.pt.

The process involved several runs comprised of the standard method for Deep Learning Model Training, which includes the gathering of several images from both India's Research Team and many Internet Databases that are accessible via APIs for later labeling in the YOLO format and consequential splitting into images with their respective labels for training and validation. Additional images were used for the detection process to test the model's effectiveness. The run's results were monitored to decide whether an increment in both the quality and quantity of images was necessary to obtain a significant improvement or whether a modification of training parameters was required to optimize the training's output. This process was repeated until a satisfactory set of Weights that met the criteria for successful detection in a virtual environment was obtained. The efficiency of the Weights in this case is, of course, validated by the remaining Training Run Data outlined in the Results Section.

4.1 Models

The Weights of Model I and Model II resulted from a YOLOv5 training with yolov5s[68] and YOLOv8 with yolov8s[69] in a Windows environment using a Desktop Computer. The subsequent Dataset mentioned in this chapter is the seven iterations of a collection of images collected and labeled for the training. Some datasets were procured from sites such as Roboflow [70], where all the images were previously labeled corresponding to their respective classes. These datasets presented several issues that affected subsequent training, due to missing or incorrect formatting. During the downloading process, many labels were either empty or didn't match the image they were supposed to represent. As a result, trials using these Datasets produced poor results. To address these issues, it was decided to create a Dataset using images from photographs and videos taken on-site by India's Research Team as part of their Investigation, which were compiled correctly and labeled.

In principle, these images will be essential for training, which could produce tailor-made results both in the primary and late stages of Project Nilbye's development. However, the poor quality, low definition, over-representation of wildlife, repetitiveness, and lack of proper identification of photographs and videos produced mixed results after the training, with this iteration of the Dataset being implemented. Examples of these problems can be seen in Figure 32. More suitable images can be seen in Figure 33.

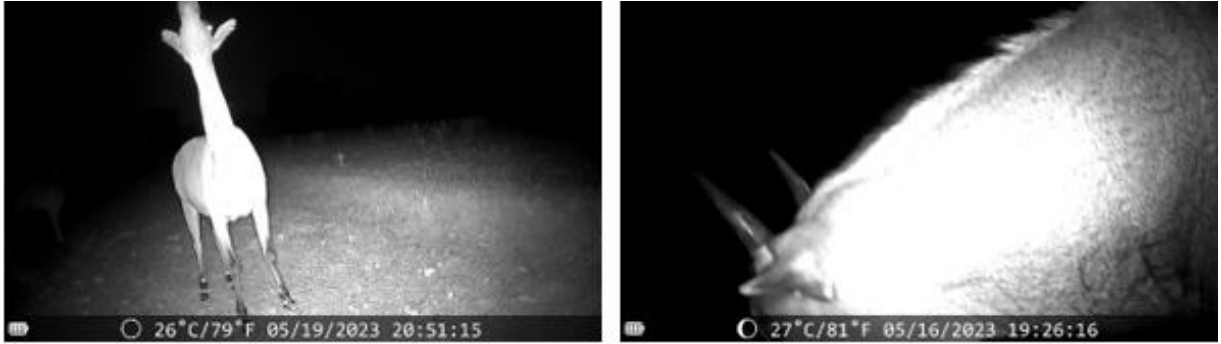


Figure 32. India's Research Team examples of inadequate images. Adapted from [4].

Considering these developments, it was decided to proceed with the concept of creating a unique Dataset, combining high-quality images taken from Internet Sites and those from India's Research Team Investigation. The remaining ones will be consequently used for further detection testing.

The Dataset described in detail in this Section is the one being used for the YOLOv5 and YOLOv8 training.

4.1.1 Dataset

For Models I & II, a total of 1329 images were gathered from India's Research Team, and stock footage from Internet sites via API request for downloading. For this Dataset, four classes were created: [0: cow, 1: nilgai, 2: water buffalo, 3: dog]. The Dataset structure aligns with the real-world conditions found in India's Team Research and Wildlife Monitoring, where classes are represented by natural occurrences as opposed to artificial balancing.



Figure 33. India's Team research image samples. Adapted from [4].



Figure 34. Internet image samples. Adapted from [71], [72].

Labeling

The images were labeled using Labelme [73], a labeling application for diverse projects related to Real-Time Object detection. Each object was labeled with a bounding box and assigned its respective class, as seen in Figure 35. The process involved no further tags as it relies only on several diverse images for a small number of classes. Due to the created labels being in JSON format, they had to be converted to YOLO format.

Splitting

The images were split to create two separate folders, train and val. The split was done following an 80/20 (training and validation, respectively) ratio. After the splitting, 1063 images were assigned to training with their respective labels, and 267 images were assigned to validation with their respective labels. A standard split for YOLO training or deep learning in general is 70/10/20, with 10 percent reserved for later testing. Given the number of images in the dataset, the 80/20 split was selected to maximize the number of training images.

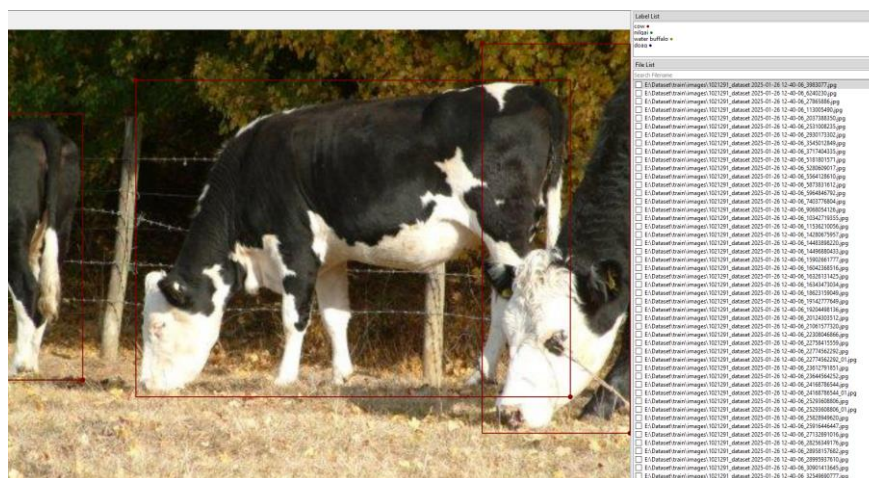


Figure 35. Example of labeling with Labelme. Adapted from [74].

The approach to creating the Dataset was to gather a diverse set of images that were meticulously curated and accurately labeled. This approach enables more efficient training by eliminating redundancies and saving time.

Labelling took considerably the most time of these tasks due to the requirement that every image possess proper Yolo-formatted labels. Automatic labeling was implemented to automate the process, but presented inaccuracies that could further hinder the Training's performance. Labeling large quantities of images with several classes can take up to 6 hours if done thoroughly. Splitting took approximately 2 hours because the images were meticulously reviewed for their potential to serve either as training or validation images. Testing material was procured separately, where some images shared similarities with the ones found on the Dataset, and other ones deviated slightly so that the future results could be better tested.

Dataset Structuring

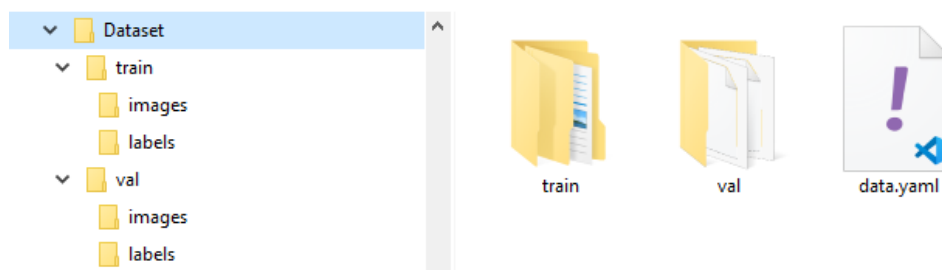


Figure 36. Dataset Overall Structure.



Figure 37. Train Structure.

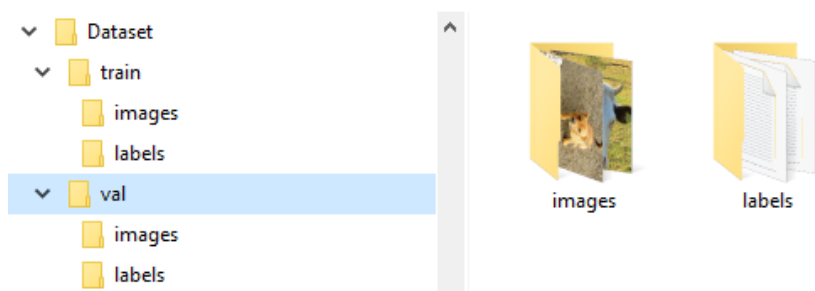


Figure 38. Validation Structure.

The data.yaml file contains the classes and the paths to the train and val folders.

```
1  names: [cow,nilgai,water buffalo, dog]
2  nc: 4
3  train: C:/yolov5/Dataset/train/images
4  val: C:/yolov5/Dataset/val/images
```

Figure 39. Data.yaml file.

4.2 Training Execution

The training was done using YOLOv5's train.py [68] and YOLOv8's CLI command yolo detect train[69] modified with command lines to use the Dataset's images and labels.

YOLOv5 and YOLOv8 were installed using Windows PowerShell. YOLOv5 was cloned from GitHub with the command git clone, and then the folder cd yolov5 was created where the requirements were installed with pip install -r requirements.txt. Later, YOLOv8 Ultralytics [69] was installed with pip install Ultralytics. YOLOv5 uses a procedural approach using Python scripts for each one of the main functions. In contrast, YOLOv8 uses a modular object-oriented approach with the yolo CLI command serving as a wrapper for executing the scripts located on the Ultralytics internal structure [75],[76].

Training Environment Description

The training and detection were developed in a Windows environment, using Windows PowerShell to execute the commands. The Desktop used for the training can be seen in Figure 40.

Windows Environment:

- Windows 10 Enterprise
- AMD Ryzen 9 5950X 16-Core Processor
- NVIDIA GeForce RTX 3080 Ti 31.0.15.5123 GPU
- 64 GB RAM
- Python 3.11
- CUDA 11.8



Figure 40. Desktop use for the Training Process.

YOLOv5 Training:

- `python train.py --img 640 --batch 16 --epochs 300 --data C:/yolov5/Dataset/data.yaml --weights "yolov5s.pt" --device 0 --project "runs/train" --name "MODEL_I" --cache --multiscale`

For the training command, it is necessary to execute `python train.py` [68], which is the script responsible for training a YOLOv5 model on a custom dataset using the `data.yaml` to access the training and validating files previously shown. The parameters set were the standard 640x640 image resolution for YOLOv5 training, a batch of 16 images that control how many images are processed at once before updating the model weights during training, a training duration of 300 epochs, and multiscale to randomly vary input image size by around 50 percent every few batches during training[68]. The training took approximately 1 hour and 30 minutes with 20 seconds per epoch approximately, fluctuating across the training.

YOLOv8 Training:

- `yolo train model=yolov8s.pt data=Dataset/data.yaml imgsz=640 batch=16 epochs=300 device=0 project="runs/train" name="MODEL_II" cache=True multi_scale=True`

The training command is necessary to execute `yolo train`, which is the subcommand of `yolo CLI` [69] responsible for executing the training on a custom dataset using the `data.yaml` to access the training and validating files previously shown. The settings are the same as in the YOLOv5 training command, being the same standard setup [69]. The training took approximately 1 hour and 15 minutes, with 15 seconds for each epoch, fluctuating across the training.

The training was done using YOLOv5's `detect.py` and YOLOv8's CLI command `yolo detect predict`, respectively, both of which were modified with command lines to use the Dataset's images and labels.

YOLOv5 Detect

- `python detect.py --weights C:/yolov5/runs/train/YOLO5_1M/weights/best.pt --source C:/test --img 640 --conf 0.25`

For the detection command, it is necessary to execute `detect.py` [68], which is the script responsible for YOLOv5 detection tasks in a set of selected images. Confidence Threshold shows only detections that have a confidence are equal and greater than 25 percent and is standard value for detection tasks [47]. The image size remains the same as in the previous training process. The detection uses Model I best. The pt results were created by the previous YOLOv5 training process.

YOLOv8 Detect

- `yolo detect predict --model C:/yolov5/runs/YOLO8_1M/weights/best.pt --source C:/test --imgsz=640 --conf=0.25`

For the detection command, it is necessary to execute `yolo detect predict` [48], which is the subcommand of `yolo CLI` [69] responsible for YOLOv8 detection tasks in a set of selected images. Confidence Threshold shows only detections that have a confidence are equal to or greater than 25 percent, which is

the standard value for detection tasks [48]. The image size remains the same as in the previous training process. The detection uses Model II best.pt results created by the previous YOLOv8 training process.

5 Software Development

A realistic detection scenario requires programming code to utilize the Model's I & II best.pt results. To this end, a main Base Code was written in Python Script to do basic detection tasks with a single input and a single output. To deal with several outputs, the Base Code was ample in the form of the Enhanced Code. This section will cover the Base Code, while the Hardware section will explain the modifications made to this script for creating the Enhanced Code and the many outputs it activates when a detection is made. This environment is the same as where the training and the detection test were conducted.

5.1 Base Code

The base code consists of a Python script that uses the Model's best.pt results from each training. The code utilizes a camera to detect images in real time with a confidence threshold in the range of 0.60 and 0.80. This threshold range was selected based on the previous detection results to balance recall and precision in a more realistic scenario for both YOLOv5 and YOLOv8 results.

There are 2 variants of this base code, one for each YOLO version that uses similar structures and workflow to load the necessary libraries, the best.pt results of each model, activate the camera, and run a Real-Time Object Detection to execute classification. These variants deviate from standard open-source applications from YOLOv5 and YOLOv8 by having tailor-made modifications for the applications needed and the detection conditions. The best.pt results can be directly used in the scripts that have been simplified to ensure a more streamlined implementation, allowing them to act with minimal user interface and permitting a more manageable structure, so both novice and experienced users can later apply modifications depending on the situation.

Programming specifications:

- CPython Architecture
- Bytecode Compilation .pyc
- Standard UTF-8 Encoding
- Automatic memory management via reference counting & cyclic garbage collector
- Global Interpreter Lock (GIL)
- Single-threaded process execution (parallelized via PyTorch if GPU acceleration is enabled).
- Dynamic Memory Allocation, Stack & Heap Segmentation
- NumPy/PyTorch tensors data representation
- Dynamical Interpreter and Just-In-Time (JIT) optimization via PyTorch when using CUDA

5.1.1 Code Overall Structure

Both codes deviate slightly, but the overall structure remains the same across the two scripts that follow a streamlined structure:

1. Camera Capture and Pre-processing Flow

1.1 Initialize Camera

- Open the webcam using `cv2.VideoCapture(0)`.
- Check if the camera is detected. If not, exit the script.

1.2 Capture Frames in a Loop

- Read frames using `cap.read()`
- If the frame is not successfully captured, print an error message and break the loop.

1.3 Preprocess the Frame

- Convert the frame to grayscale.
- Apply bilateral filtering to reduce noise.
- Convert to LAB color space and apply CLAHE (Contrast Limited Adaptive Histogram Equalization) for contrast enhancement [77].
- Use sharpening filters to improve feature visibility.
- Apply gamma correction for brightness adjustment.

2. Object Detection and Display Flow

2.1 Load the YOLO Model

- YOLOv5 variant uses `DetectMultiBackend(weights_path, device=device)`.
- YOLOv8 variant uses `model = YOLO(model_path)`.

2.2 Process the Frame for YOLO Detection

- Convert the frame to the correct format:
 - YOLOv5 requires resizing to (1280, 1280) and converting to a PyTorch tensor.
 - YOLOv8 directly processes the frame.

2.3 Filter and Draw Bounding Boxes

- Apply Non-Max Suppression (NMS) to remove duplicate detections.
- Retrieve detected objects and their confidence scores.
- Draw bounding boxes around detected objects.
- Display labels with class names and confidence scores.

2.4 Show Detection Output

- Display the processed frame in a window using `cv2.imshow()`.
- If the user presses 'q', exit the loop and close the window.

The key differences between both versions are how they handle image processing. At the same time, the Model I variant takes extra steps for resizing and tensor conversion before sending the frame to the YOLOv5.pt results and applying NMS (Non-Max Suppression) manually, the Model II variant can process frames directly without explicit tensor conversion and can handle NMS directly due to its built-in API before sending them to the YOLOv8 best.pt results [78]. For images to be suited for YOLO models, they need to be converted into tensor, which is a multi-dimensional array that is optimized for GPU computation [79]. It is important to understand the role of the following libraries present in the script variants:

For Model I & II:

- **Torch:** Is Pytorch's core Python package that provides tensor operations, neural network layers, and CUDA support [80].

For Model I only:

- **Numpy:** Handles numerical computations in Python, supporting the use of large dimensional arrays and matrices with the addition of many other mathematical functions [81].

In Model, I variant script, `DetectMultiBackend`, and `non_max_suppression` modules are controlled by Pytorch, while `scale_boxes` is controlled by both libraries.

For Model II only:

- **Ultralytics:** The Ultralytics library and its YOLO class contain the built-in tools essential for key functions such as inference, image processing, and weight formatting. These built-in tools are the following [82]:
 - **Model classes (YOLO):** Allows for the best.pt results to be uploaded.
 - **Training utilities:** These utilities enable training and fine-tuning for YOLOv8 models.
 - **Validation and export tools:** Allows inference for various formats (Pytorch, ONNX [83], TensorRT [84], etc).
 - **Inference Processing:** Handles image processing.

YOLOv5 variant Flowchart for Model I best.pt

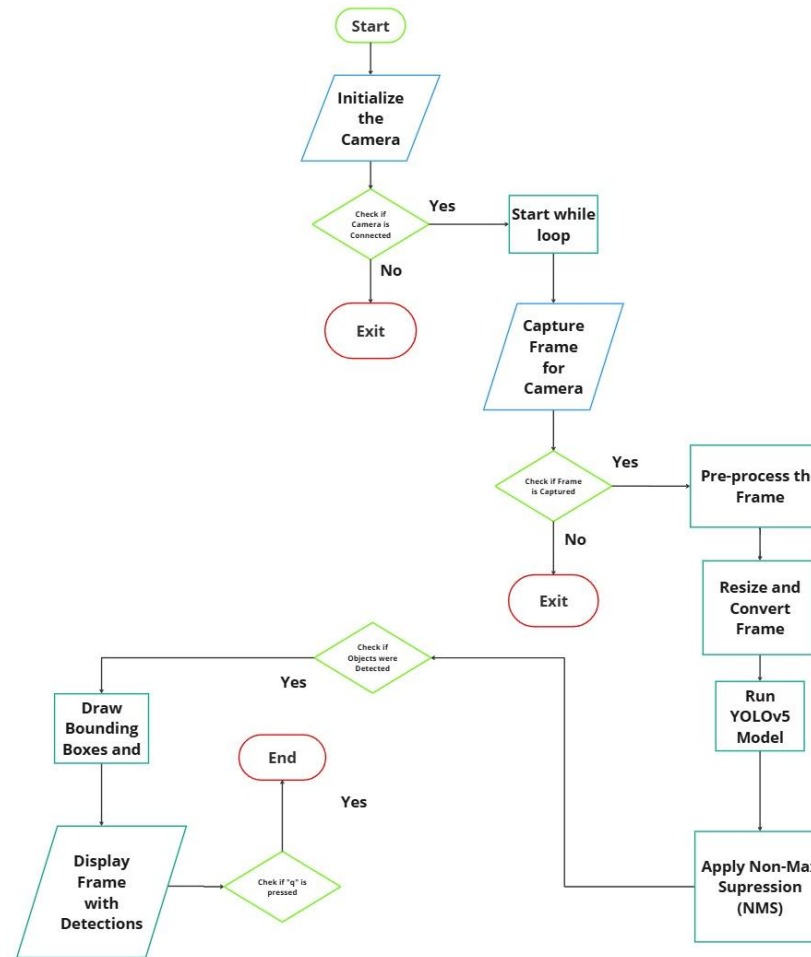


Figure 41. Model I Base Code Variant Flowchart.

The complete code can be found in the Appendix.

YOLOv8 variant Flowchart for Modell II best.pt

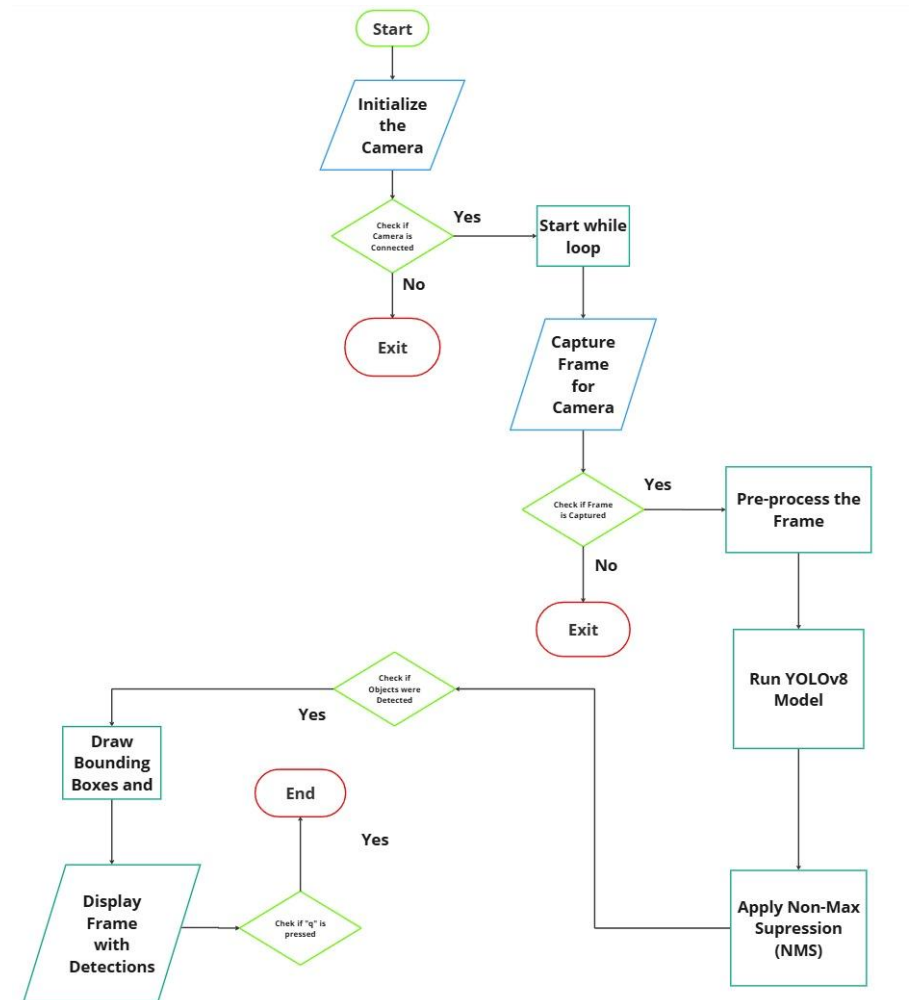


Figure 42. Model II Base Code Variant Flowchart.

The complete code can be found in the Appendix.

5.1.2 Rapid Detection Test with YOLOv5 and YOLOv8 variants for Model I & II best.pt

The Model I & II Base Code variants need a USB camera to perform detections. The detection test was made in the same Windows environment and with the same CPU mentioned in the previous Section. The testing setup can be seen in Figure 44.

For these tests, an HD 1080p EMEET was used, as seen in Figure 43.



Figure 43. USB EMEET camera.

The reason for using this simple USB camera is to carry out a rapid test with both YOLO version Models.pt results on real photographs in a stationary position that doesn't consider major distance or movements. This task proves to be too simple for working with the Dahua PTZ Camera, which will be tested using the Enhanced Code variants for both YOLO versions. This rapid detection test also offers the possibility to observe how best.pt results behave differently with different hardware, such as cameras. The results of this rapid detection test are found in the Results Section. In both Model I & II Variants, the confidence threshold was set to be equal to or greater than 0.80 to keep false positives or unwanted detections due to environmental factors in the test conditions as minimal as possible.



Figure 44. Setup for rapid detection test.

Inference

The Enhanced Code was written to be implemented using the NVIDIA Jetson Orin Nano. To upload Models, I & II best.pt results into this code; it was necessary to convert them into another format that will be compatible with the NVIDIA Jetson Orin Nano Linux environment. The format selected was ONNX.

YOLOv5's uses export.py [68] and YOLOv8's yolo export [69] CLI command.

Exporting for YOLOv5

```
python export.py --weights C:/yolov5/runs/train/ MODEL_I /weights/best.pt --include onnx --simplify --dynamic
```

For the detection command, it is necessary to convert the best.pt YOLOv5 results into best.onnx using the export.py[47]. The simplified setting is used to optimize the ONNX model by making the computation graph smaller, cleaner, and faster. In contrast, the dynamic setting enables dynamic input shapes when exporting to ONNX format [47].

Exporting for YOLOv8

```
yolo export model=C:/yolov8/runs/train/MODEL_II/weights/best.pt format=onnx simplify=True dynamic=True
```

For the detection command, it is necessary to convert the best.pt YOLOv8 results into best.onnx using the yolo export subcommand of CLI [48]. The simplified setting is used to optimize the ONNX model by making the computation graph smaller, cleaner, and faster. In contrast, the dynamic setting enables dynamic input shapes when exporting to ONNX format [69].

5.2 Enhanced Code

The Enhanced Code is the Model I (YOLOv5) and Model II (YOLOv8) Base Code Variants, modified in key aspects such as:

- Enabling communication with the Dahua PTZ Camera via IP address and adapting the settings for optimal streaming. This is done to replace the HD 1080p EMEET used for the Base Code variants, which will allow for widening the range, improving the image resolution and quality, and controlling the functions outlined in Section 3.
- Replacing the Model I & II best.pt results respectively with the best.onnx results for inference with the Jetson Nano Orin in the Linux Ubuntu Environment.
- Adding the time and serial libraries to allow the Python script to communicate and control an additional Arduino sketch. The reasoning for adding these features will be explained in this Section on 5.3.4 Demonstration Device. The Arduino code can be found in the Appendix Section.

YOLOv5 variant Flowchart for Modell II best.pt

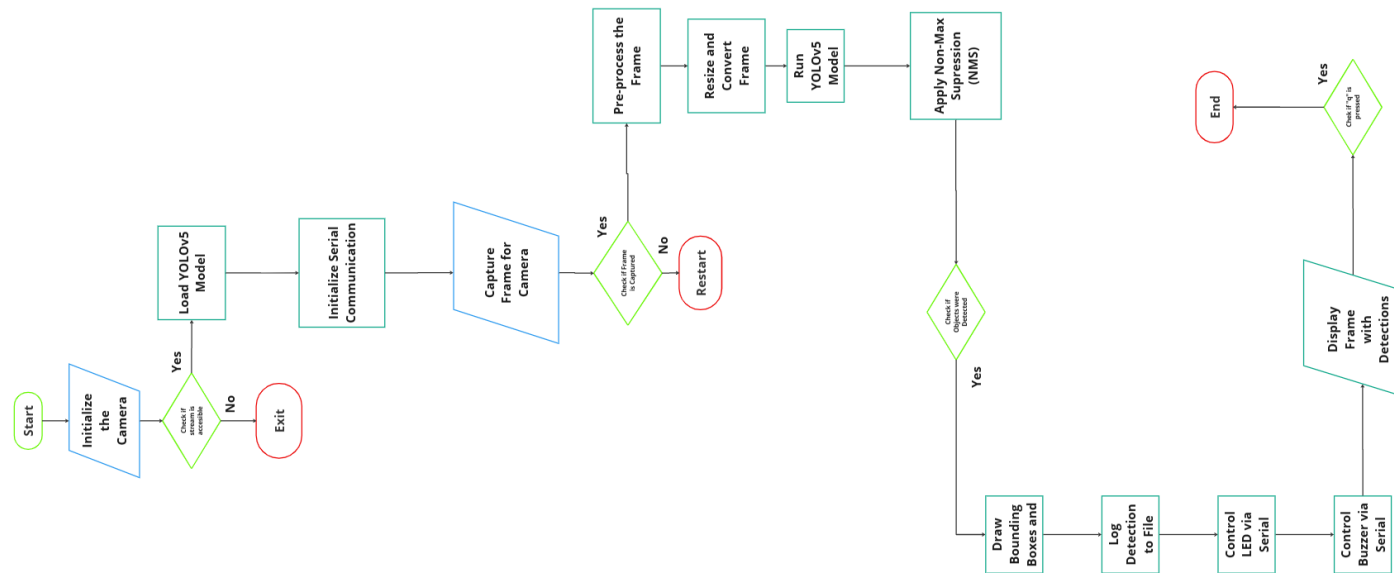


Figure 45. Model I Enhanced Code Variant Flowchart.

The complete code can be found in the Appendix

YOLOv8 variant Flowchart for Modell II best.pt

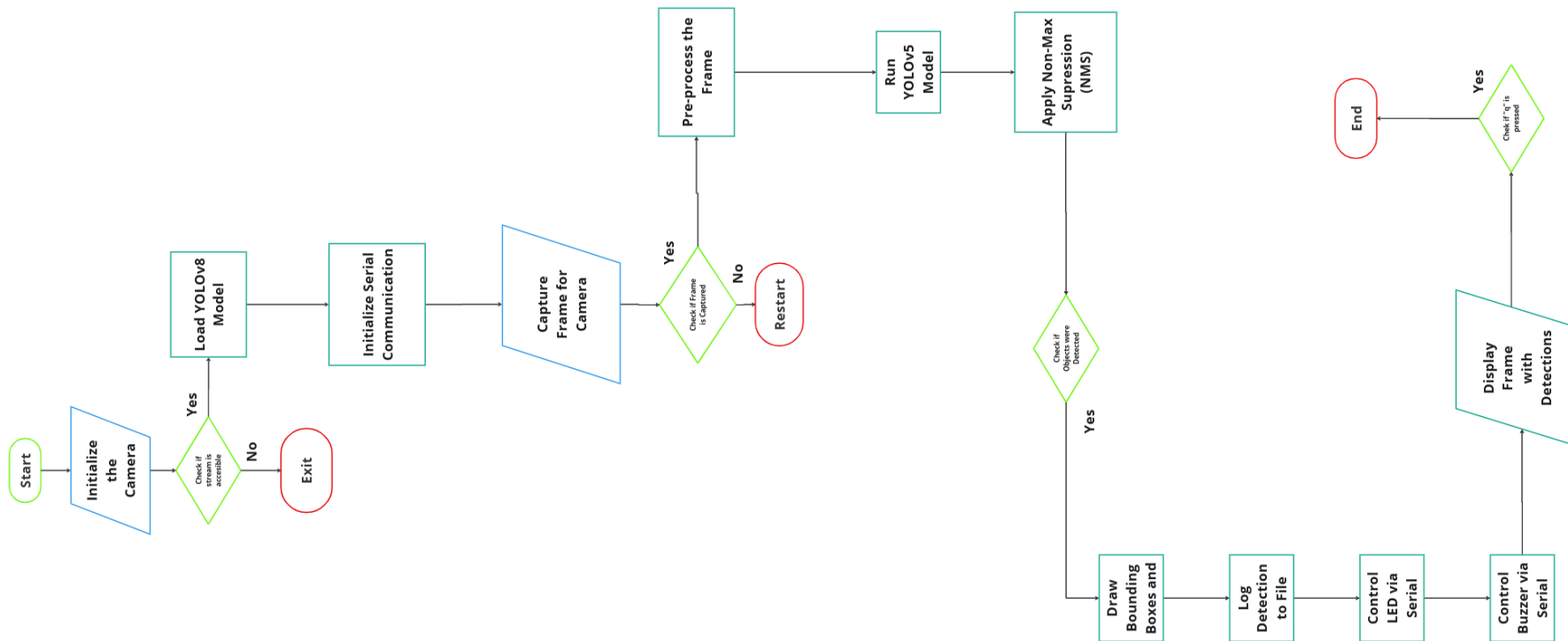


Figure 46. Model II Enhanced Code Variant Flowchart.

The complete code can be found in the Appendix.

The main goal of these updates and upgrades, is to enable the code to use several functions that activate additional outputs. The conditions for correct detection of an object classification, such as a Confidence Threshold equal to or greater than 0.80, remain the same as in the Base Code variants mentioned previously in this Section. The image processing function remains the same as well.

5.3 Demonstration Device

An experimental setup was assembled to simulate the future Project Nilbye's Prototype Device on a smaller scale. The following devices and components were assembled to create the demonstration device:

1. NVIDIA Jetson Orin Nano Developer Kit
2. SanDisk Extreme 128 GB microSD card
3. Arduino Mega Elegoo 2560 R3
4. USB Cable
5. Dahua PTZ Camera
6. Bretboard
7. Blue LED
8. 220 Ohm Resistance
9. Passive Buzzer x2
10. Ultrasonic Sensor
11. PIR Sensor
12. Jumper Cables M-F & F-M



Figure 47. Arduino Mega Elegoo 2560 R3.



Figure 48. Blue LED.



Figure 49. Passive Buzzer.



Figure 50. 220 Ohm Resistance.



Figure 51. USB Cable.



Figure 52. Breadboard.

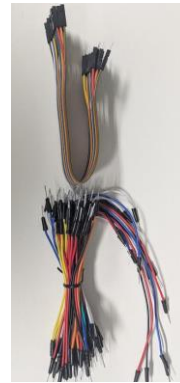


Figure 53. Jumper Cables M-F & F-M.

The Jetson Nano Orin is connected to the Arduino Mega 2560 R3 (Elegoo) via a USB Type-B to USB-A cable. Through this connection, the Jetson communicates with the Arduino using UART serial communication protocol. This setup enables the Jetson Orin Nano connects to control and activate an LED connected in parallel to a 220 Ohm resistor, two buzzers, and the PIR and ultrasonic sensors.

Due to the transition to Jetpack 6.x and upstream kernel changes, typical libraries such as Jetson.GPIO became depreciated and alternatives like gpiod require significant complexity and inconsistency while configuring the GPIO pins as reliable digital outputs [85],[86],[87].

As a result, direct hardware control with the Jetson Nano Orin 40-pin Header was not feasible. Therefore, it was decided to create this demonstration device using the Jetson Nano Orin Developer Kit to handle high-performance tasks and the Arduino board to execute the low-level hardware control. The Jetson Orin Nano also interfaces with an HD 1080p EMEET USB Camera and, in future detection tests, the Dahua PTZ Camera. The microcomputer is also connected to other peripherals, such as a monitor via an HDMI cable, with the addition of a mouse and a keyboard. The complete setup can be seen in Figure 54.

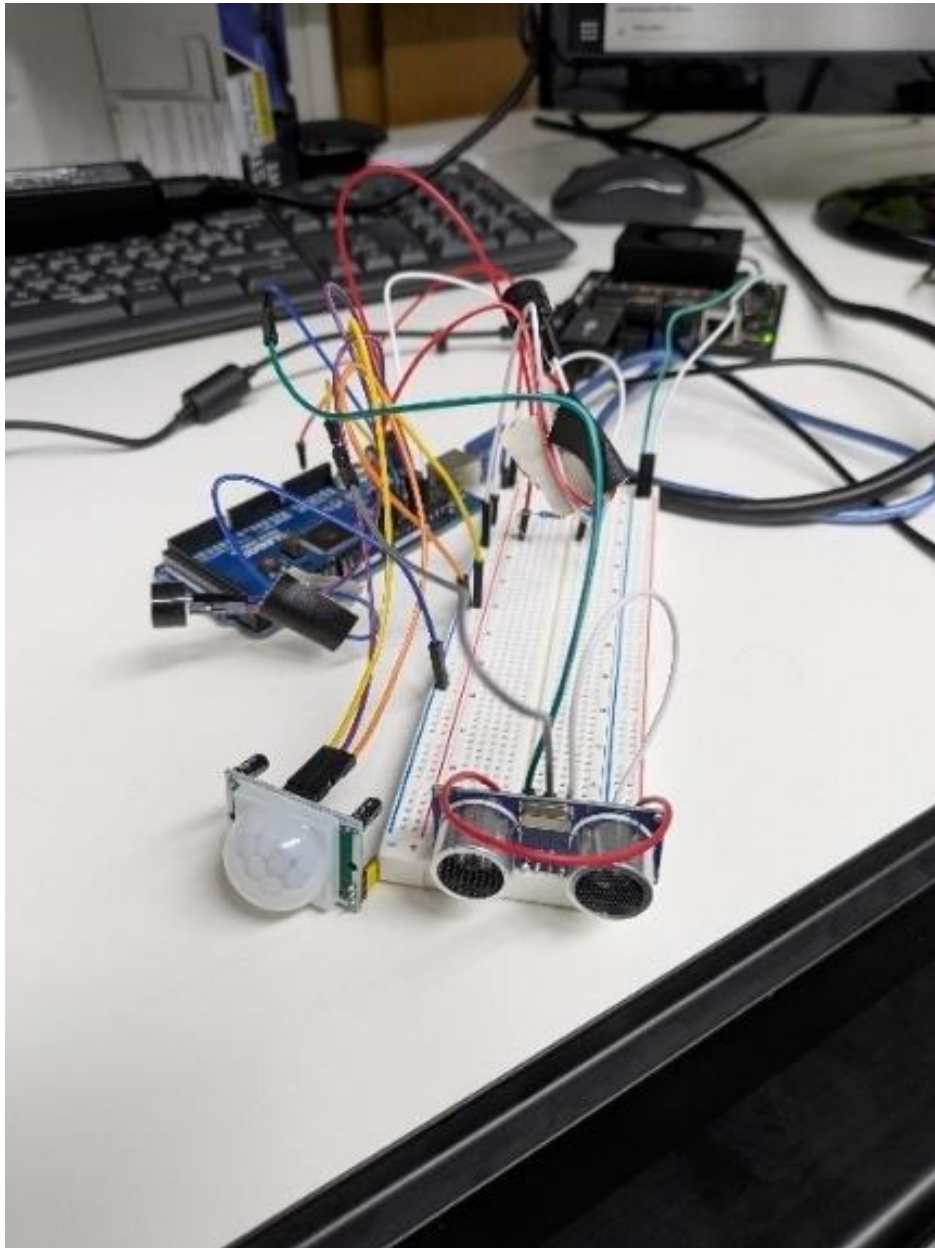


Figure 54. Demonstration Device networking.

Ideally, the goal would be to directly control all components with the Jetson Orin Nano 40-pin GPIO Header with its usual interfaces, and this is why the Demonstration Device is meant to represent the system's outputs that are equivalent to the ones found on the original Prototype Device in Section 3. The connections can be seen in Figure 55. The LED represents the blue light and how it will be activated when an object is detected in real-time. In the case of the demonstration is also meant to validate the output in the case of true detection, while the two buzzers have different frequencies to represent the loudspeaker and the ultrasound. Therefore, these frequencies are different to differentiate how the response will be if an object comes into the inner boundary of the device detection 2nd layer.

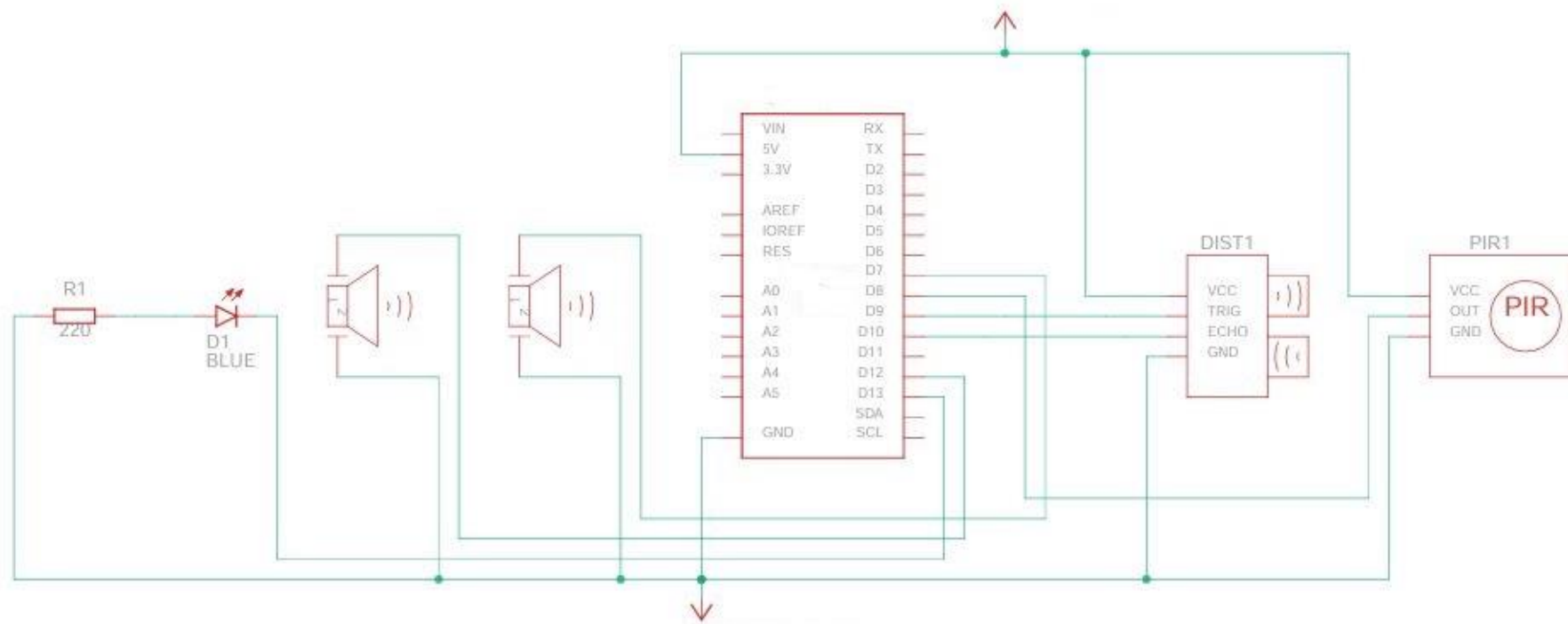


Figure 55. Arduino board connection diagram

5.3.1 Advanced Test with YOLOv5 and YOLOv8 variants for Model I & II best.pt

To execute the final test, the Demonstration Device controlled by the NVIDIA Jetson Orin Nano was integrated with the Dahua PTZ Camera previously listed in Section 3. The complete network can be seen on the left side of Figure 56. The purpose of upgrading the system with this camera is to test the response considering two important factors: distance and dynamic placement. Real photographs will be placed both outside and inside the boundaries set by the PIR and Ultrasonic sensors used in the Demonstration Device, so that the camera can detect them in different positions. This response will allow the observation of the Model I & II best.onnx results in non-stationary positions with dynamic placement included.

The results obtained from this test will serve as an approximation of the system's future behavior, considering environmental factors such as lightning, brightness, depth, and background objects.

The figure on the right illustrates one of the test examples, where a real photograph is placed progressively closer to the camera within its line of sight to observe the system's reaction in both detection and simulated deterrence using the LED and the buzzers, aided by the PIR and Ultrasonic sensors.

The Dahua PTZ Camera was mounted on a wall, as shown in Figure 56, near the Demonstration Device. For enabling communication with both the NVIDIA Jetson Nano Orin and the Demonstration Device, the camera needs to be paired with both. Therefore, all the devices that require access to the internet, whether via Ethernet or WIFI, need to be connected to the same network. This was achieved by connecting the camera via LAN cable to a Fritz!Box Router and the NVIDIA Jetson Nano Orin via Wi-Fi. The Arduino Board needs to be connected only via USB to the NVIDIA Jetson Nano Orin, so it can control the components as previously mentioned. The camera was configured using Dahua's ConfigTool, as seen in Figure 57, on the Windows Environment described in Section 4, including the same PC Desktop, which was also connected to the same network.

Once the camera was configured, the connection via IP address with the NVIDIA Jetson Nano Orin was enabled with the use of the Enhanced Code for both Models I & II, as shown in this Section. To this end, the camera's IP and credentials (User ID and Password) are used on Model's I & II Enhanced Script variants to access via stream with the use of RTSP_URL.

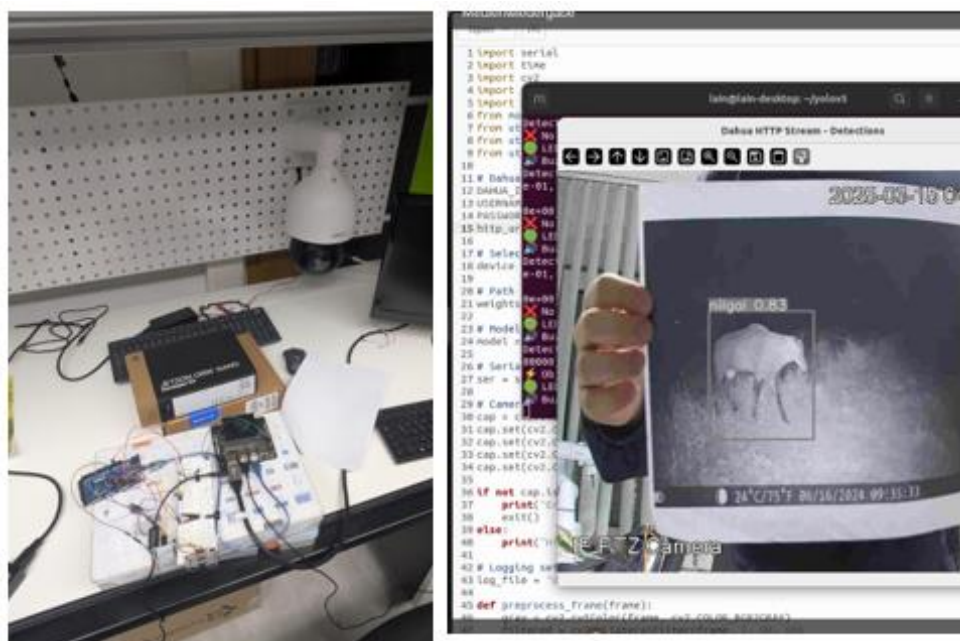


Figure 56. Setup for advanced detection test.

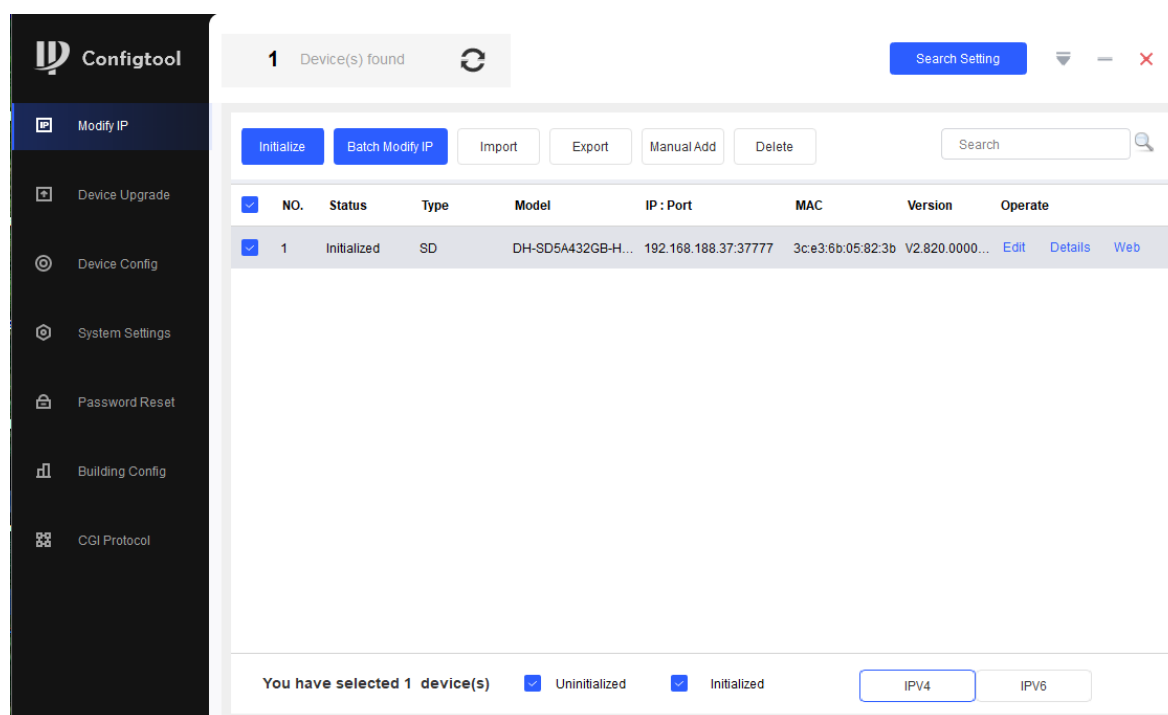


Figure 57. Dahua ConfigTool.

6 Results

The training and detection results are presented separately in this Section for both the YOLOv5 and YOLOv8 Models (I and II, respectively). The training results values are training and validation loss for boxing, object, and classification and metrics such as precision, mean average precision, F1 score, and accuracy. The detection results values are the detection rate of Models I & II when being tested with images in both digital and printed in DIN A4 paper sheets. The detection rate is measured by the number of images correctly detected and classified. This measure applies to the simple and rapid detection tests where there is a single input->output scheme. For the advanced detection test, aside from the detection rate, the system's additional outputs that represent the responses of the LED and the buzzers listed in Section 5 are also considered as results to measure the system's response.

6.1 Training Results

- **Box Loss:** Measures the error in predicting the bounding box locations. Lower values indicate better localization of objects. This metric is based on IoU (Intersection over Union) as shown in Equation 4, which measures overlap between the predicted and ground truth bounding boxes [52],[88].

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Equation 4. Intersection over Union Formula. Adapted from [51],[89].

- **Objectness Loss:** Measures the confidence of an object being present in a bounding box. Lower loss means the model is more confident in its predictions [89].
- **Classification Loss:** Evaluates how well the model classifies detected objects into correct categories. Lower values indicate better classification accuracy [89].
- **DFL Loss (YOLOv8-specific):** Distillation Feature Loss, unique to YOLOv8's anchor-free approach, helps refine bounding box predictions for improved localization [89].
- **Precision:** The proportion of detected objects that are correct, as illustrated in Equation 5. A high precision means fewer false positives (incorrect detections) [52],[88].

$$Precision = \frac{TP}{TP + FP}$$

Equation 5. Precision Formula. Adapted from [51],[89].

- **Recall:** The proportion of real objects that were correctly detected, as demonstrated in Equation 6. A higher recall indicates fewer missed detections (false negatives) [52],[88].

$$Recall = \frac{TP}{TP + FN}$$

Equation 6. Recall Formula. Adapted from [51],[89].

- **F1-Score:** The harmonic mean of Precision and Recall, balancing detection accuracy and completeness, seen in Equation 7 [88].

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Equation 7. F1-Score Formula. Adapted from [89].

- mAP@50 (Mean Average Precision at IoU 50%): Measures how well the model ranks detections, assessing localization accuracy at a 50% IoU threshold [52],[88].
- mAP@50-95: A more challenging metric averaging mAP across IoU thresholds (50% to 95%), indicating how well the model generalizes [88].

YOLOv5 Results

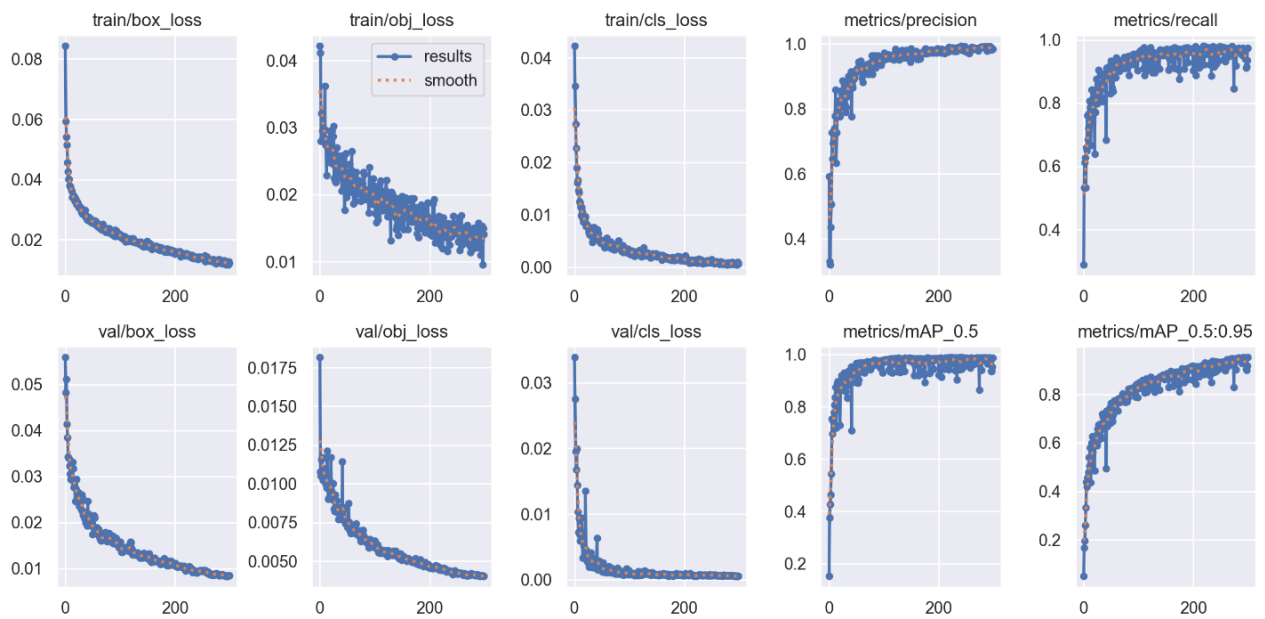


Figure 58. YOLOv5 Training Results.

In Figure 58, the YOLOv5 Training Metrics show a strong learning curve, with all relevant loss functions (box, objectness, and classification) decreasing consistently over time. Precision, Recall, and mAP@5 improved rapidly during the early epochs and stabilized at high values.

As seen in Figure 59, the confusion matrix of YOLOv5 presents high classification accuracy for all object classes, with Cow and Water Buffalo having the highest results (99% each). In outlier cases, Nilgai and Water Buffalo could be confused with the background.

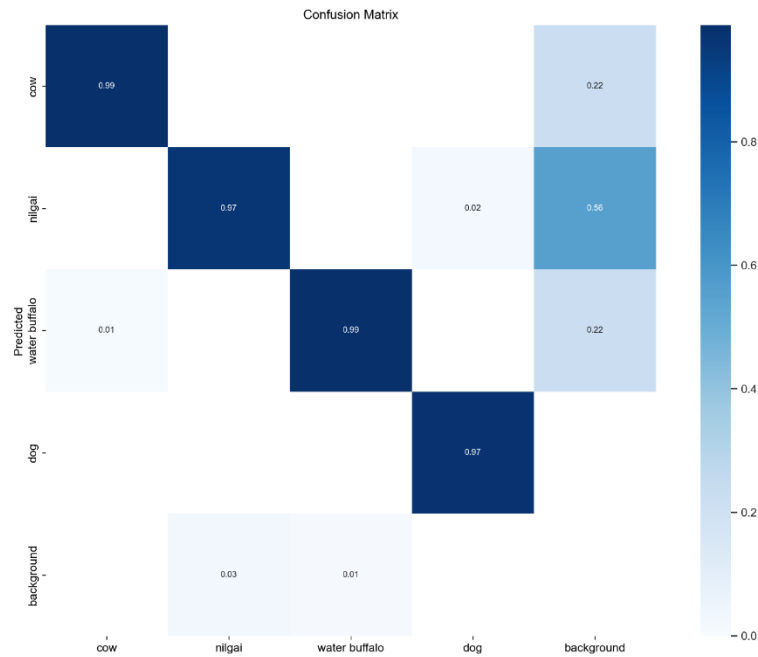


Figure 59. YOLOv5 Confusion Matrix.

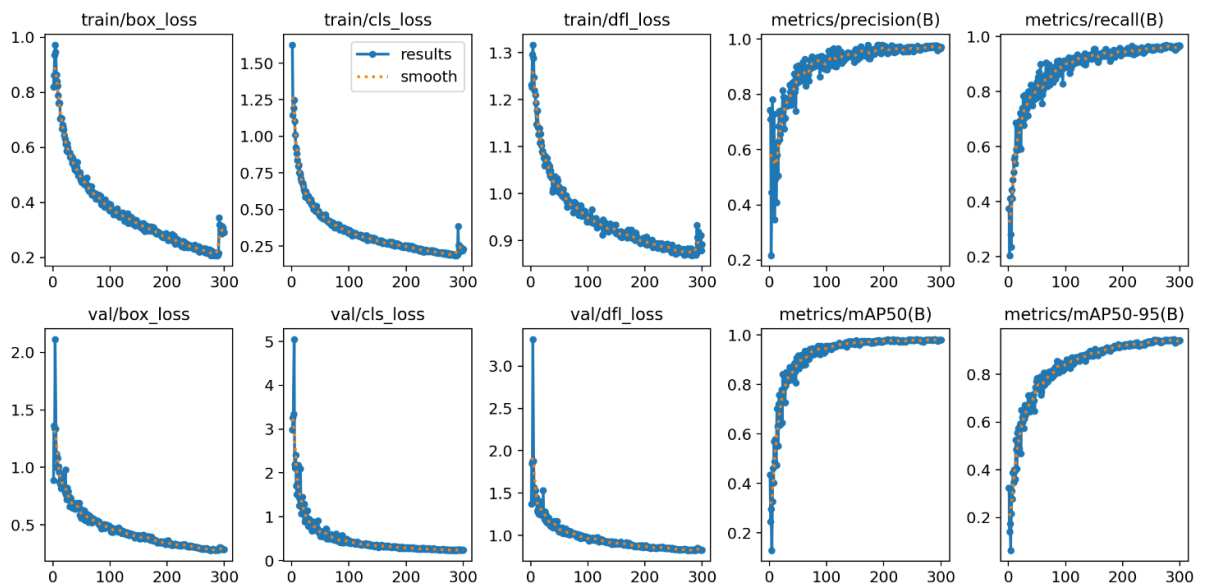


Figure 60. YOLOv8 Training Metrics.

For the YOLOv8 Training Metrics seen in Figure 60, demonstrate a more gradual decline in comparison to YOLOv5, while Precision, Recall, and mAP@5, steadily increase and stabilize throughout the training.

YOLOv8 Results

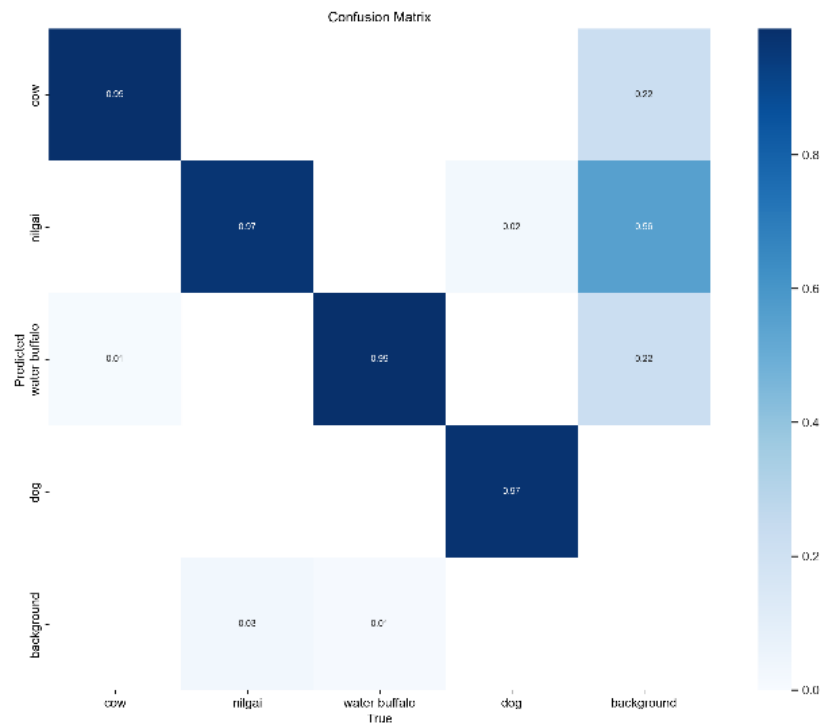


Figure 61. YOLOv8 Confusion Matrix.

The YOLOv8 Confusion Matrix in Figure 61, similarly to the YOLOv5 Confusion Matrix, illustrates a strong classification performance, with a high count of true positive instances for all class objects. Nevertheless, there is still a tendency to misclassify Nilgais as background more frequently than in the YOLOv5 example.

Training Metrics Comparison YOLOv5 vs YOLOv8

Model	Precision	Recall	F1-score	mAP@50
YOLOv5	1.0	0.99	0.98	0.987
YOLOv8	1.0	0.98	0.97	0.982

Table 3. YOLOv5 vs. YOLOv8 Training Metrics comparison.

While YOLOv5 and YOLOv8 show very similar metrics such as Recall, mAP@0.5, Precision, and F1 Score, their performance and approach to detection differ due to possible misclassifications, especially in possible real-world scenarios, where YOLOv8 outperforms YOLOv5 by a small margin, due to this model's ability to apply better class separation.

6.2 Detection Results

6.2.1 Simple Detection Results

From 68 tested pictures, Model I (YOLOv5) was able to detect and correctly classify 66 images. Three of them were partially correct due to instances of false positives. The remaining two pictures were complete false positives. From the correctly classified images, only 38 were equal or greater to the 0.90 confidence threshold. Figure 62 presents examples of correct classification with partial and complete false positives.

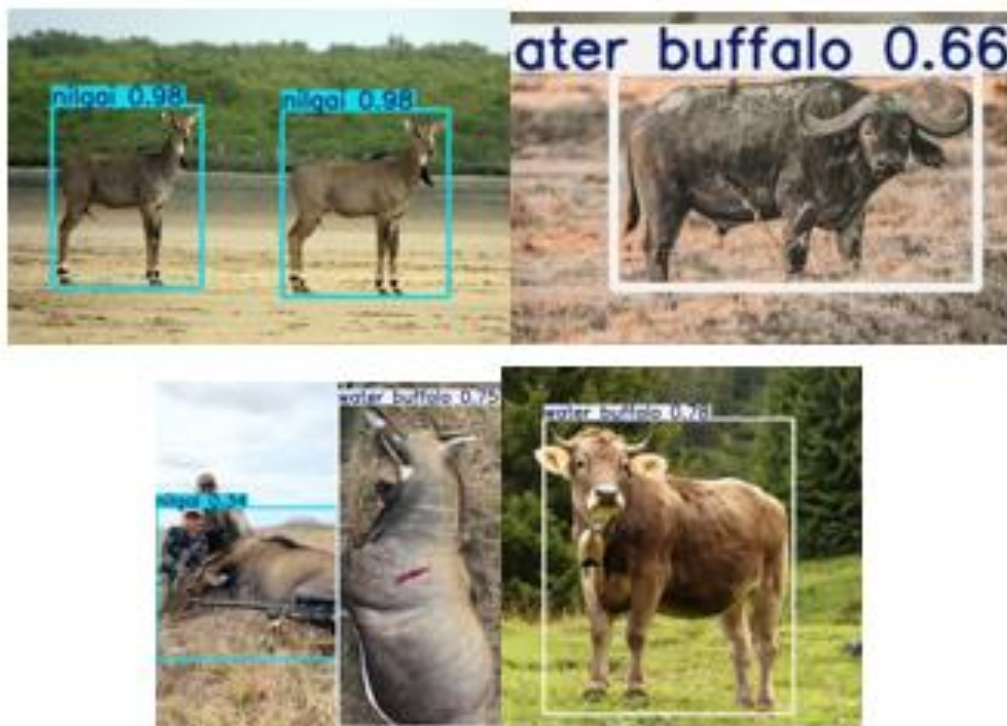


Figure 62. Examples of correct classifications with partial and complete false positives from left to right and top to bottom, respectively. Adapted from [90],[91].

From the same batch of images, Model II (YOLOv8) detected and classified 63 images, one of them was partially correct due to instances of false negatives, and 4 presented correct classification with false positives. The remaining 5 pictures were complete false positives. From the correctly classified images, only 41 were equal or greater to the 0.90 confidence threshold. Figure 63 presents examples of correct classification and partial and complete false positives with the addition of false negatives.

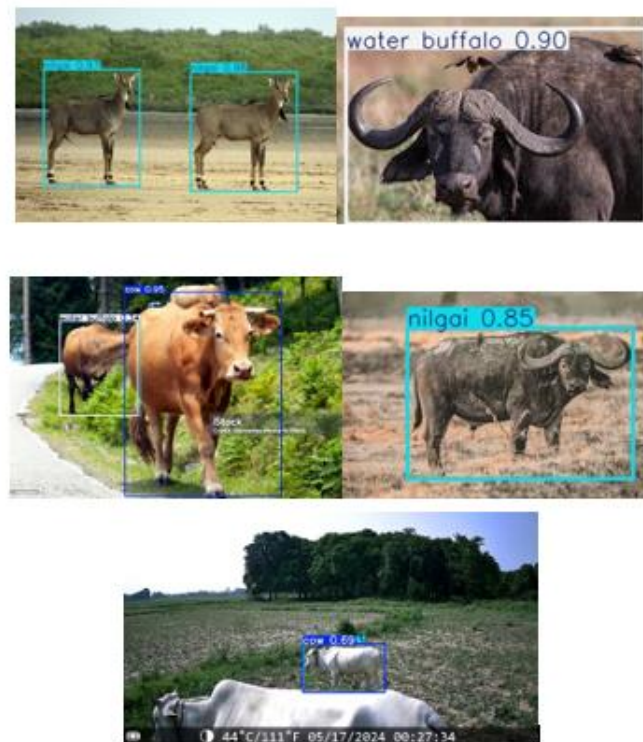


Figure 63. Examples of correct classifications with partial and complete false positives from left to right and top to bottom, respectively. Adapted from [90],[91],[5].

Detection Results Comparison YOLOv5 vs YOLOv8

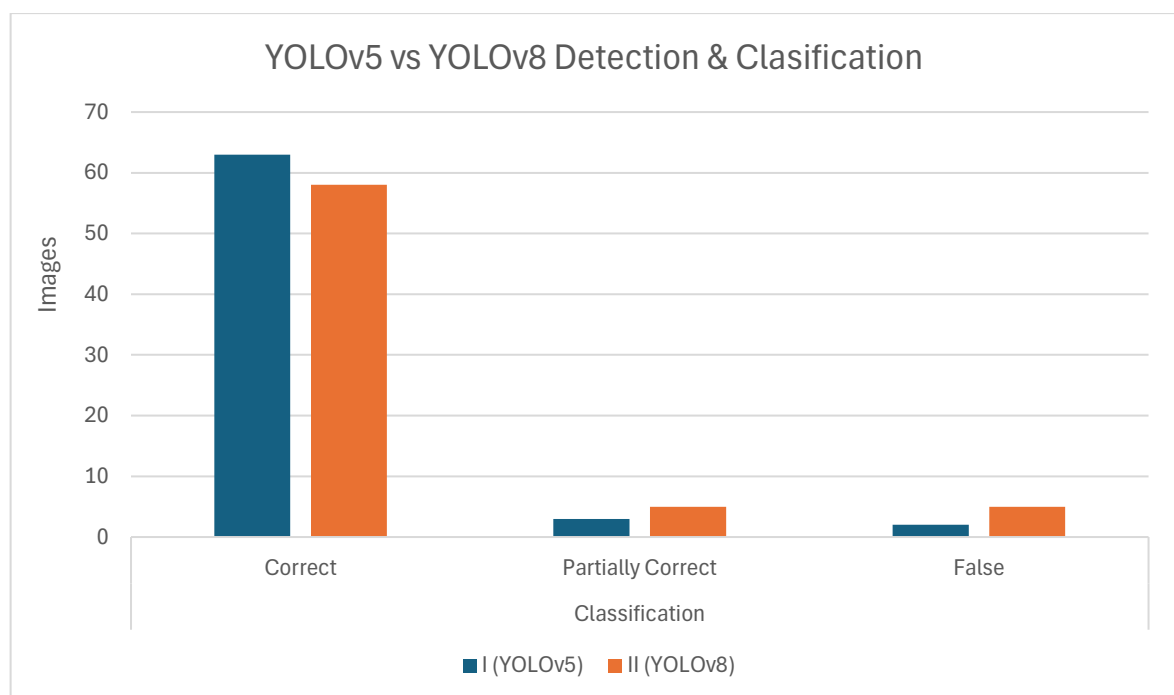


Figure 64. YOLOv5 vs YOLOv8 Detection Comparison Chart.

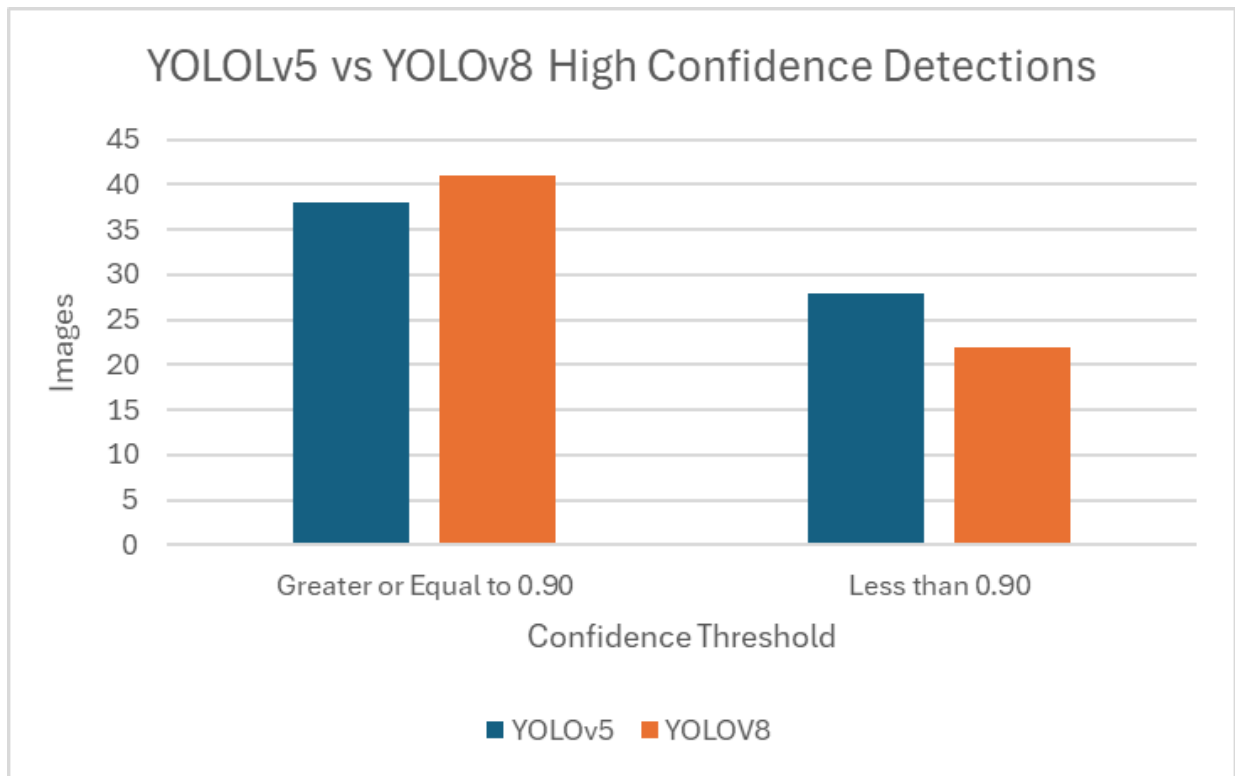


Figure 65. YOLOv5 vs YOLOv8 High Confidence Comparison Chart.

6.2.2 Rapid Detection Results

The same 68 batch images were printed and tested. The correct detection and classification criteria were proper identification immediately when placed in front of the camera. If further adjustments were needed for the camera to detect the photograph, then it was considered a false positive due to the latency.

The Model I (YOLOv5) was able to detect and classify 46 photographs correctly. There were 3 false positives, and the remaining 19 were false negatives. Out of the 46 correctly classified photographs, 29 had a confidence threshold equal to or greater than 0.90. Figure 66 shows examples of correct classification, false negatives, and false positives.

The photographs look slightly different due to the image processing function (see Base Code for Model I in the Appendix), which serves to improve detection accuracy. The removal of the function didn't alter the results.



Figure 66. Examples of correct classification, false negatives, and false positives from left to right, from top to Bottom respectively. Adapted from [90],[91].

In comparison with the previous detection test for Model I (YOLOv5), the detection rate had a 29 % decrease considering all the correct detections, both complete and partial, while the presence of false positives had a 1 % increase, and the rate of classified images with a confidence threshold equal or higher to 0.90 had a 6 % increase.

As for Model II Base Code Variant (YOLOv8), detection and classification for 48 images were correct. There were 11 false positives, and the remaining 9 photographs were false positives. The images below show examples of correct classification, false negatives, and false positives. Of the 48 correctly classified photographs, 38 had a confidence threshold equal to or greater than 0.90. Figure 67 shows examples of correct classification, false negatives, and false positives.



Figure 67. Examples of correct classification, false negatives, and false positives from left to right, from top to Bottom respectively. Adapted from [90],[91].

While doing minimal effect for this Base Code variant, the image processing function (see Base Code for Model I in the Appendix) still adds accuracy to the results, and its removal doesn't affect the results.

In comparison with the previous detection test, the Model II (YOLOv8) detection rate showed a 23 % decrease, considering all the correct detections, both complete and partial. The presence of false positives had a 16 % increase, and the rate of classified images with a confidence threshold equal to or higher than 0.90 had a 19 % increase.

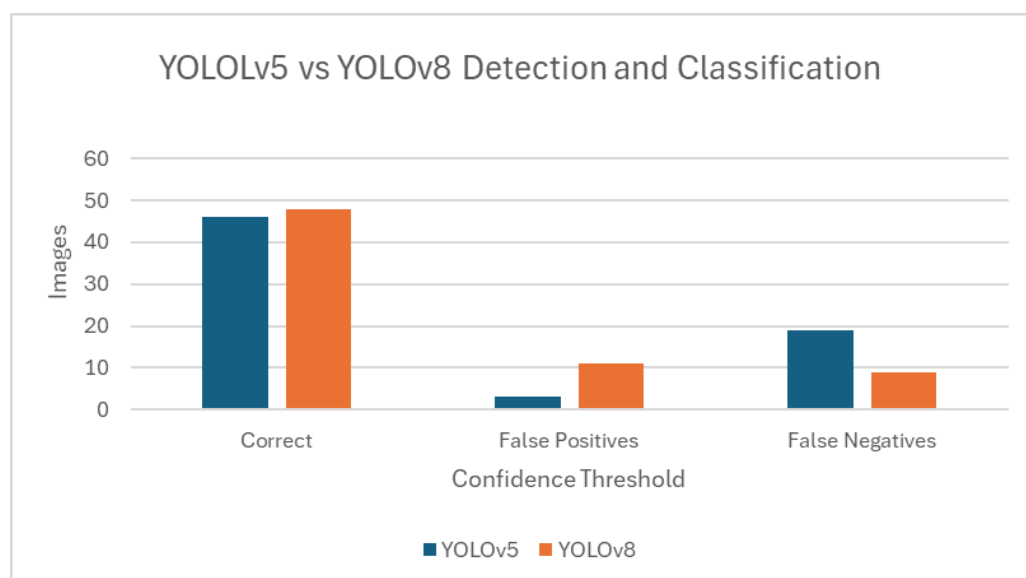


Figure 68. YOLOv5 vs YOLOv8 Detection Comparison Chart.

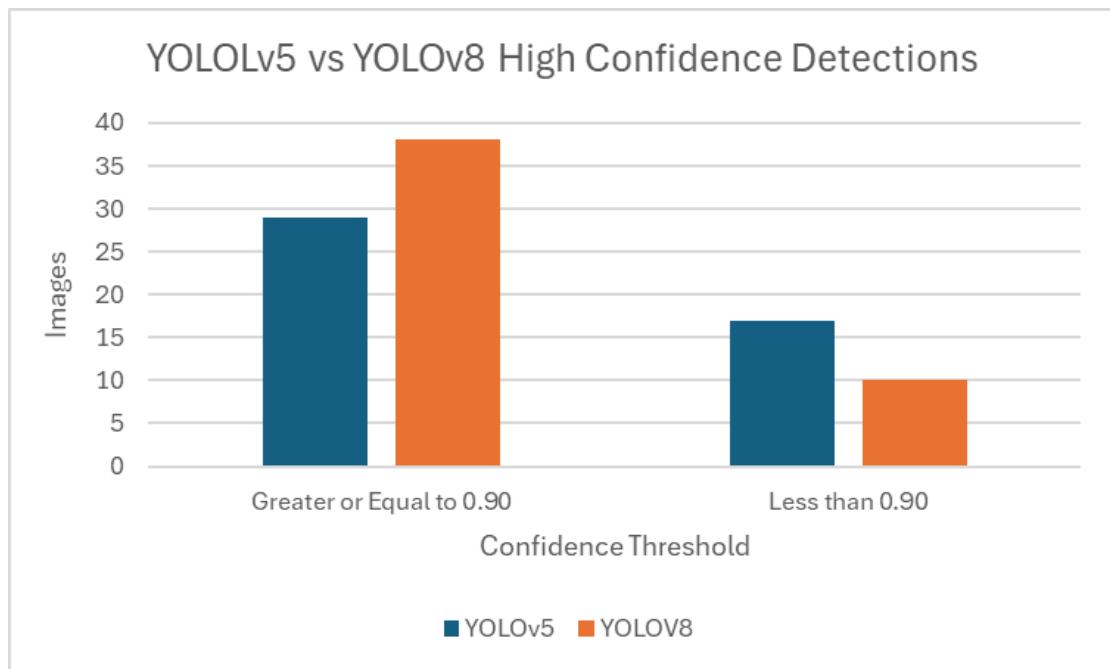


Figure 69. YOLOv5 vs YOLOv8 High Confidence Comparison Chart.

6.2.3 Advanced Detection Results

Due to the conditions for dynamic placing, 10 images from the original batch were for testing. These images were chosen due to their detection rate, high confidence threshold values, and proper resolution. The latter is a decisive factor, given that low-quality images are discarded for testing that involves significant distance. This fact was not an issue in previous tests and allows for further analysis of Models I & II best.pt results in different images with varying quality levels and resolution that differ from the original training settings, such as the ones outlined in Section 4. The criteria for correct detection and classification were the same as the rapid detection test in this Section, with the added boundaries set by the PIR and Ultrasonic sensors as parameters for the detection process. The correct labeling and classification were set to take place both in the inner and outer boundaries. If further adjustments are needed for the camera to detect the photograph, then it is considered a false positive due to the latency. Further responses to a correct detection and classification from the Demonstration Device are also considered to test the system's output accuracy further.

The Model I variant (YOLOv5) identify 5 photographs correctly. The remaining 5 were false negatives. In addition, there were 5 false positives due to the presence of background objects on the testing site. All the correctly classified photographs had a confidence threshold equal to or greater than 0.80. Figure 70 shows examples of correct classification.

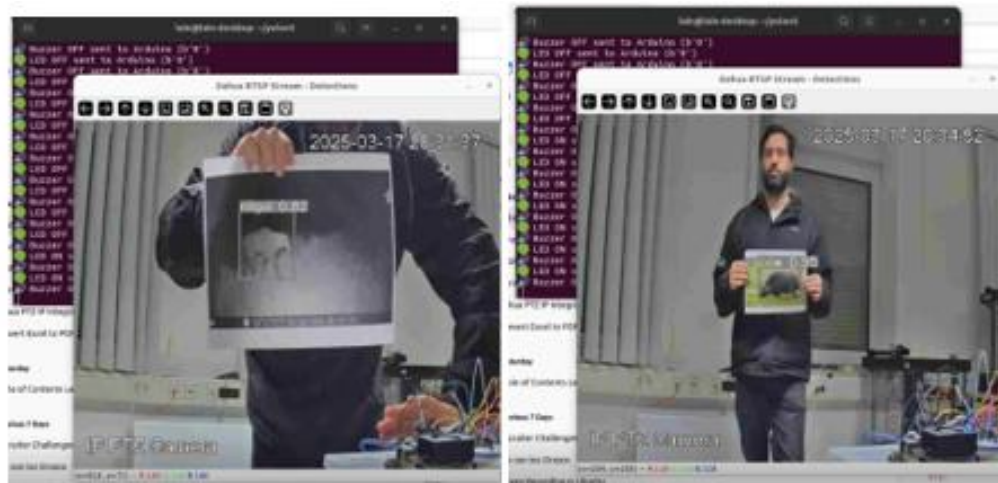


Figure 70. Examples of correct identification.

Every time the photographs were correctly identified, the Demonstration Device managed to deliver the proper simulated response by emitting low and high frequencies with the buzzers and activating the LED. This behavior happened only when there was detection and classification. However, the response at times was erratic because the frequency didn't always correspond to the distance where the object was placed. This tendency was present for both correct and false classification.

As for the Model II variant (YOLOv8), the test was inconclusive due to the lack of correct detections and false ones as well. The current settings, further discussed in Section 7, didn't allow the Model to perform in any capacity, hindering its ability to identify any of the objects. Due to this lack of detection, the system wasn't able to deliver any response. Given this outcome of Model (YOLOv8), further testing was made connecting the HD 1080p EMEET USB Camera directly to the NVIDIA Jetson Nano Orin and testing the original 68 image batch. The results remain consistent with the rapid test made in Section 5 for both Models I & II (YOLOv5 and YOLOv8, respectively). In both cases, dynamic placement posed a significant challenge, and both models were unable to perform detection over greater distance. In all detection cases that included positive and negative classification, the system responded with the proper outputs of LED and buzzers. Figure 71 shows examples of correct classification using the USB camera from previous tests.



Figure 71. Examples of correct identification using the EMEET USB Camera.

7 Results Analysis

The training and detection results are discussed separately in this Section, while also comparing both Models I & II (YOLOv5 and YOLOv8, respectively) to reach an overall conclusion and determine which version is better for the Prototype Device's main task and what other purposes each version can better fulfill. These results will also serve to determine the effectiveness of the training process outlined in Section 4 and the detection tests made in Section 5, where the influence of both settings and present conditions found in these experiments are greatly considered and weighed to obtain a greater insight into the causes for the given results.

7.1 Training Results

The conditions for the training listed in Section 4 were optimal, meeting the requirements for this experiment, and the commands used were set to standard, including resolution, epochs, and confidence threshold. The following analysis proves that the results obtained reflect this fact.

The steady decline in loss and the consistent increase in performance along the training epochs, shown in the figures, represent an effective learning process for both YOLOv5 and YOLOv8 Models. While these trends tend to be gradual, especially from the early epochs, the convergence over time indicates that the Models achieved their goal of learning the object's classification with minimal inaccuracies and false positives. The rest of the metrics also showed very similar behaviors.

The precision of 1.0 suggests a very low tendency for false identification, as both Models can avoid incorrect detections. The difference between Recall implies a higher advantage for YOLOv5 to detect more objects, meaning that YOLOv8 misses a few more (an actual object that exists in the image and belongs to a class the model was trained to detect). F1-Scores favor YOLOv5 slightly due to a marginal edge in detection balance, while YOLOv8 performs at a high level in this regard. Mean Average Progression (mAP@50) is slightly higher for YOLOv5, which can translate into more precise bounding boxes than YOLOv8 and, therefore, a marginally stronger detection consistency. Finally, accuracy is strong for both, suggesting fewer mistakes from YOLOv5 compared to YOLOv8.

Both YOLOv5 and YOLOv8 perform at an exceptionally high level with perfect precision and high recall, F1-score. While YOLOv5 has a slight edge in recall, the F1-score differences are minimal. The results suggest that YOLOv5 might be slightly better at detecting all objects without missing any, whereas YOLOv8 could have advantages in speed, efficiency, or generalization. The confusion matrices reveal specific misclassification trends. YOLOv5 achieves higher per-class accuracy, particularly in correctly identifying cow and water buffalo, with fewer overall misclassifications. In contrast, YOLOv8 misclassifies nilgai as background in 56% of cases, which could explain its slightly lower recall. Additionally, YOLOv8 has a higher tendency to mistake background objects for real detections, contributing to its marginally lower accuracy.

7.2 Detection Results

The conditions for the rapid and advanced detection tests listed in Sections 4 & 5, respectively, were not idyllic due to false positives caused by background objects and the presence of environmental factors, such as lighting. While problems caused by the former were resolved with the adjustment of the confidence threshold, the same could not be said for the latter, despite the best efforts to adjust the hardware settings and the testing site conditions to improve the performance. The photographs used for testing Model I & II Base and Enhanced Code for the Variants proved a significant challenge despite the training and image processing settings established in Sections 4 & 5. These difficulties persisted across various test iterations and hardware configurations, including the HD 1080p EMEET and Dahua PTZ cameras.

When the first test was done with both YOLOv5 and YOLOv8 commands, as described in Section 4, both Models I & II had a high performance with slight inaccuracies, proving that even with a small Dataset and standardized training settings, correct detection and classification were achievable. However, the steep decline between consequent tests demonstrated that the results (mostly insulated with the training process), didn't translate into an optimal Real Time Object Detection due to both Models I & II struggles with dynamic placing, environmental conditions, and background objects, along with the inability to overcome the challenge of greater distance detection, even with the implementation of advanced equipment. Model I (YOLOv8) was the most affected by this decline, being the last test inconclusive for this variant, and making the comparison between the Model's performance inconclusive as well. Several factors can be related to all the mentioned test behaviors, such as:

- Problems with streaming while using the Dahua PTZ Camera. These difficulties were the product of lowering image quality by using a sub-stream rather than the Main one with a significantly reduced framerate. These adjustments were made to avoid stream crashes or image freezing and enable the detection process. This was the main cause of Model II's low performance.

This Performance and inconclusive results due to YOLOv8's high requirements are listed in Section 3.

- The testing material chosen proved to be extremely difficult for both types of cameras. Given the lack of solid object testing samples, which in this case will be wildlife, it proved to be a complicated endeavor to determine how accurately the system can detect and classify objects in real-time with the use of photographs. Both cameras presented problems with image quality and framerate, meaning that many images needed to be placed in an exact, undetermined position to be detected; there were other environmental factors and backgrounds that either led to false positives or simply created obstacles for the detection process.

However, Model II's (YOLOv8) underperformance provides greater insight into Model I's (YOLOv5) consistency despite the detection rate decline throughout the different testing scenarios. Model I also struggles with low image quality due to the adjustments of the video streaming, but it still manages to produce a decent performance, showing great potential for future implementation. Model II could

reach this performance level and even outperform Model I, but it proves that its requirement needs better quality settings, which are not always possible to achieve.

The system output's behavior was independent of classification and relied more on the photograph's image placement, while not being real solid objects, proved challenging for both PIR and Ultrasonic sensors to accurately measure the given distance from the Demonstration Device, making a precise assessment of the system's performance difficult. This development also highlighted potential flaws and, therefore, further refinement for the Enhanced Code Variants of Model I & II main functions was required.

The high contrast between the rapid and advanced tests has shown a bigger difference between the results of hardware implementation than the YOLOv5 vs YOLOv8 performance. While the Dahua PTZ Camera is more advanced than the HD 1080p EMEET USB-Camera, it ultimately displayed little difference when using both Model I & II, and even when this shortcoming is most likely the result of an unstable internet connection along with lower quality image and video settings, which only furthers the need for higher requirements. The HD 1080p EMEET USB Camera, on the other hand, manages to produce the same results and, in some cases, exceed the ones produced with the Dahua PTZ Camera, relying on fewer requirements and even achieving greater Real Time Object detection.

7.3 Overall Summary

The contrast between the training metrics and the testing results shows the significance of hardware implementation and environmental conditions, presenting a necessity for further refinement of the training process settings, image processing methods, and low reliance on sophisticated networks and advanced equipment to solve problems such as greater distance detection, especially for smaller objects and dynamic image placement. The YOLOv5 and YOLOv8 comparison remains ultimately inconclusive, mostly due to YOLOv8's inability to present concrete results on the last testing scenario. However, YOLOv5, under reliance on advanced equipment as mentioned in Section 3, demonstrates its real potential for deployment, while YOLOv8 optimal training results offer potential for an even better performance, in comparison to YOLOv5, given that the problems related to video streaming and network connectivity are solved. With this analysis, the Model I (YOLOv5) variant aligns better with Project Nilbye's requirements and presents a more practical setting for optimization and deployment, making it the better choice.

8 Recommendations

The training and subsequent detection tests have left room for significant improvement while also presenting a wide range of potential scenarios that require diverse solutions, which are not covered in this study. Therefore, this Section attempts to construct a framework capable of providing future improvement in the form of recommendations based on the shortcomings present throughout the entire experimental process due to constrictions that hindered the system's performance and influenced the obtained results. The focus of these recommendations will be placed on the training process and the future deployment of the system as part of Project Nilbye's development. This approach is based on considering how the contrast between training metrics and detection results can be better utilized to improve the system's performance.

8.1 Improving Training

The number of images presented in the Dataset, as seen in Section 4, was the result of an effort to fulfill the necessary data requirements and overcome all the challenges presented, resulting in a deliberate choice of quality over quantity. Complete representation could not be fully achieved due to reduced instances for one class; in this case, dogs. This issue is not the sole cause of Model I & II's subsequent inaccuracies, but it remains vital to find a balancing solution regarding this matter. As for a complete solution to tackle the problems of working with limited data using a time-consuming process necessary to ensure its quality, it is recommended to deviate from standard training by adjusting the learning rate using a cosine annealing scheduler or slightly lowering the rate of the initial value (e.g., 0.001 $\rightarrow$ 0.0005) to prevent overfitting early on and enable mosaic and mix-up augmentation. Additionally, the Confidence Threshold should be lowered to allow more detections, reduce false negatives, and adjust the IoU threshold to better detect smaller objects. Finally, using GAN (Generative Adversarial Network)-based augmentation tools like Stable Diffusion can allow a more realistic diversity in the Dataset. GAN is a type of Deep Learning Network that allows for the generation of new data similar to the one found in the Dataset [92].

But overall, the best improvement comes from continuous training where, depending on the object's size being detected, different YOLOv5 and YOLOv8 model variants such as (nano, small, medium, large, and extra-large) are selected, and many iterations of the training made while validating them with detection tests in real-time rather than relying solely on training metrics. YOLOv8 is recommended as the primary model for the training, given its improved architecture, faster inference speed, and efficient training pipeline, which includes native support better suited for new augmentation techniques and updated loss functions [53]. YOLOv5, on the other hand, could be used as an alternative in cases where compatibility or inference speed is prioritized over marginal gains in accuracy.

Some of the measures that were taken on subsequent training done as part of this study were left out due to their inconclusiveness or marginal contribution to the outcome of the results. This fact doesn't

completely invalidate their effectiveness, given that the implementation could not be further improved due to time constraints and the significant effort required for this task. Another reason for these measures being presented as recommendations rather than being implemented in further testing is the previous inexistence of a given pipeline for the experimental process, which is one of this Bachelor's Thesis's main goals. The training process, along with the establishment of testing conditions and hardware setup, took a considerable amount of preparation outside of the actual execution time, which only allowed for a reduced number of Datasets to be prepared and training iterations to be executed, being the ones presented in this Bachelor Thesis the best results achieved.

Given the importance of continuous training, these recommendations will overlap with the rest of the process for Project Nilbye implementation and deployment.

8.2 Improving for Project Nilbye

Remaining faithful to the Project's requirement, images from the India Research Team were used to create the Dataset. However, these images lack any labeling or proper identification, consequently carrying inaccuracies through the entire pipeline from labeling to detection testing. These shortcomings highlight the need for a dedicated team solely for continuously procuring high-quality on-site images in large amounts by photographing the objects, in this case, local wildlife. Another second team would handle labeling meticulously, whether it is manually, semi-automatically, or fully automatically. The first team should explicitly identify the objects in the images and assign clear instructions for labeling to the second team. Both teams should be in constant and retroactive communication for the image quality and quantity to be improved and for the proper labeling to be confirmed. These measures will ensure the creation of better datasets, fit for any training process.

Following the second recommendation phase for Project Nilbye: continuous detection and testing. As in the training process, the detection tests should be a continuous and retroactive endeavor where a third team is responsible for constantly upgrading Models I & II Enhanced Code Variants and maintaining the Prototype Device. This third team should report their findings to the other teams for continuous improvement in the training process to be made. This coordinated approach can ensure the creation of a reliable pipeline and a final Prototype Device that aligns with the Project's Goals.

As for improving the detection testing process itself, an experimental proposal is outlined to tackle the challenges found during this study, such as reducing detection with greater distance and object size, along with environmental conditions and background objects. The experimental set consists of a constricted space similar to a shooting range gallery, where the Prototype Device replaces the shooter, so that the camera can be pointed in a single direction to detect and classify objects placed dynamically at many distances. The testing will still be done using real photographs, but, in this case, they will have real and small sizes where the large cutouts will be used to simulate objects far away in the outer boundaries, and small images can be used to simulate objects in the inner boundaries. The dynamic placing will also help to determine how the system can accurately respond to different distances, leading to a better refinement of the Model's I & II Enhanced Code Variants.

Given the dimensions of a closed shooting gallery, this setup will be an upgrade from the one practice in this study, as explained in Section 5, which is not expected to deliver ultimate results on the Model's I & II performance. The conditions of this setup will discard potential scenarios and create a more fitted testing environment without the need to use real-life subjects, reducing the potential risks of interacting with wildlife. This closed environment will also allow for the third team to recreate several scenarios with different environmental conditions where lightning can be switched, depth adjusted, and background objects can be placed in different positions to change parameters and avoid false positive detections. The "Project Nilbye's Detection Gallery," as seen in Figure 72, can either be constructed on-site or in a single location, being either state-of-the-art or locally makeshift, following a set of requirements that include a closed or semi-closed tunnel environment.

The first, second, and third teams can also be comprised of local volunteers to further strengthen the relationship between Project Nilbye and the communities it aspires to help, while at the same time acquiring a better knowledge of the present conditions. This is recommended to address the issue found in the last detection test in Section 5, where advanced hardware relies on sophisticated infrastructure for both power and internet connectivity. Lastly, for the Prototypes Devices' design, it is important to strike a balance between potential but often non-existent results with advanced technologies, and more practical and repeatable outcomes with systems that combine high- and low-tech components and devices. This goal must be achieved, aligning with Project Nilbye's original goals of sustainability and cost-effectiveness.

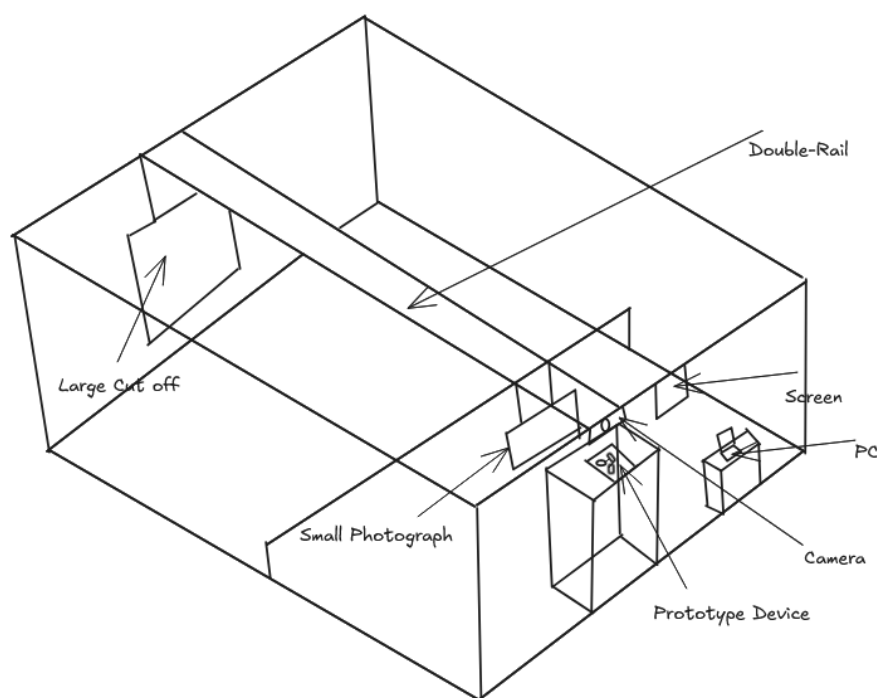


Figure 72. Project Nilbye's Detection Gallery.

Conclusion

While the goals of this Bachelor's Thesis to document and summarize the experimental process of the Back End Software Development have been achieved, there is still a longer path to achieving the development of the Prototype Device. The study and analysis of all major aspects related to the training process and detection testing aim to offer the necessary tools and an optimal framework for the Werk:Raum team to continue iterating and improving on the foundations established to materialize the Project's milestones in the foreseeable future. Even with some inconclusive results, the study can offer precise metrics as a frame of reference for new iterations, as well as possible scenarios that can accurately predict future events when deploying the Deep Learning Models created after the training.

This is especially true if the recommendations outlined in the results analysis are taken into consideration during both the experimental process and the development of Project Nilbye. In addition, the end of the study added further insight into the potential mishaps of favoring advanced technological solutions and sophisticated algorithms, given that the detection testing results proved this by showing the system's ability to perform better while working with limitations rather than achieving greater potential with the implementation expensive high technological devices and complex networks, even due the potential to better utilize these tools with continuous improvement of all the development steps exists.

The research and development for this Bachelor's Thesis also shed light on how the current state of Real Time Object Detection is constantly evolving. From individual DIY projects to full-scale industrial enterprises with significant resources, the field is incredibly diverse. Both ends of this contribute to expanding the collective knowledge and fostering the development of new applications. Nevertheless, it is also important to highlight the importance of further iteration and improvement without reliance and complacency with every new advancement related to Artificial Intelligence, since this framework progresses as far as researchers' efforts in both academia and data analysis allow it.

In this new technological frontier that, this body of work aspires to modestly contribute to, by further developing the technical aspects regarding software of the Project Nilbye, so that it can reach its maximum potential, and, in the future, this innovative endeavor can position itself as a pioneering example of sustainable solutions, to address major environmental problems.

Bibliography

- [1] T. H. Würzburg-Schweinfurt, "Werk:Raum :: Technische Hochschule Würzburg-Schweinfurt," Technische Hochschule Würzburg-Schweinfurt - Zur Startseite mit Accesskey 0. Accessed: Mar. 24, 2025. [Online]. Available: <https://www.thws.de/forschung/institute/idee/idee-projekte/werkraum/>
- [2] Welthungerhilfe and Technische Hochschule Würzburg-Schweinfurt, "Pioneering Farm Protection through Wildlife Movement Prediction." 2023.
- [3] "Partners • Network & Collaborations - Rural Water Supply Network." Accessed: Mar. 25, 2025. [Online]. Available: <https://www.rural-water-supply.net/fr/partners/32-welthungerhilfe>
- [4] Welthungerhilfe and Technische Hochschule Würzburg-Schweinfurt, "Coexistence between Small Farmers and Wildlife: Identification-Based Acoustic Deterrents." 2023.
- [5] Welthungerhilfe and Technische Hochschule Würzburg-Schweinfurt, "Field Testing for Addressing Crop Damage by Wildlife." 2023.
- [6] "AI vs. Machine Learning: The Key Differences and Why They Matter," Lifewire. Accessed: Mar. 20, 2025. [Online]. Available: <https://www.lifewire.com/artificial-intelligence-vs-machine-learning-8782221>
- [7] "Machine Learning vs Traditional Programming: Choosing the Right Approach for Your Projects | Institute of Data." Accessed: Mar. 20, 2025. [Online]. Available: <https://www.institutedata.com/us/blog/machine-learning-vs-traditional-programming-choosing-the-right-approach-for-your-projects/>
- [8] "What is Machine Learning?," GeeksforGeeks. Accessed: Mar. 20, 2025. [Online]. Available: <https://www.geeksforgeeks.org/ml-machine-learning/>
- [9] "Machine Learning vs. AI: Differences, Uses, and Benefits," Coursera. Accessed: Mar. 20, 2025. [Online]. Available: <https://www.coursera.org/articles/machine-learning-vs-ai>
- [10] "Differences: Machine Learning vs Artificial Intelligence - GAMCO, SL." Accessed: Mar. 25, 2025. [Online]. Available: <https://gamco.es/en/machine-learning-vs-artificial-intelligence/>
- [11] "AI vs. Machine Learning vs. Deep Learning vs. Neural Networks | IBM." Accessed: Mar. 20, 2025. [Online]. Available: <https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks>
- [12] M. A. Wani, F. A. Bhat, S. Afzal, and A. I. Khan, *Advances in Deep Learning*, vol. 57. in *Studies in Big Data*, vol. 57. Singapore: Springer, 2020. doi: 10.1007/978-981-13-6794-6.
- [13] M. A. Wani, F. A. Bhat, S. Afzal, and A. I. Khan, *Advances in Deep Learning*, vol. 57. in *Studies in Big Data*, vol. 57. Singapore: Springer Singapore, 2020. doi: 10.1007/978-981-13-6794-6.
- [14] A. Sujak, D. Jakubas, I. Kitowski, and P. Boniecki, "Identification of Factors Affecting Environmental Contamination Represented by Post-Hatching Eggshells of a Common Colonial Waterbird with Usage of Artificial Neural Networks," *Sensors*, vol. 22, no. 10, Art. no. 10, Jan. 2022, doi: 10.3390/s22103723.
- [15] "DEEP LEARNING, THE FINAL STAGE OF AUTOMATION AND THE END OF WORK (AGAIN)?," *Psychosociological Issues Hum. Resour. Manag.*, vol. 5, no. 2, p. 154, 2017, doi: 10.22381/PIHRM5220176.
- [16] "Deep learning definition, algorithms, models, applications and advantages | Science online." Accessed: Mar. 25, 2025. [Online]. Available: <https://www.online-sciences.com/robotics/deep-learning-definition-algorithms-models-applications-advantages/>
- [17] X. Jiang, A. Hadid, Y. Pang, E. Granger, and X. Feng, Eds., *Deep Learning in Object Detection and Recognition*. Singapore: Springer, 2019. doi: 10.1007/978-981-10-5152-4.
- [18] M. M. Taye, "Understanding of Machine Learning with Deep Learning: Architectures, Workflow, Applications and Future Directions," *Computers*, vol. 12, no. 5, p. 91, Apr. 2023, doi: 10.3390/computers12050091.

- [19] H. Alhichri, Y. Bazi, N. Alajlan, and B. Bin Jdira, "Helping the Visually Impaired See via Image Multi-labeling Based on SqueezeNet CNN," *Appl. Sci.*, vol. 9, no. 21, Art. no. 21, Jan. 2019, doi: 10.3390/app9214656.
- [20] T. S. Huang, "Computer Vision: Evolution and Promise".
- [21] object Object, "Kohteentunnistus kuvista konvoluutioneuroverkoilla", Accessed: Mar. 25, 2025. [Online]. Available: <https://core.ac.uk/reader/92854840>
- [22] "Deep Learning for Generic Object Detection: A Survey | International Journal of Computer Vision." Accessed: Dec. 03, 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s11263-019-01247-4>
- [23] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," *Multimed. Tools Appl.*, vol. 82, no. 6, pp. 9243–9275, Mar. 2023, doi: 10.1007/s11042-022-13644-y.
- [24] "2D Object Detection: A Survey." Accessed: Mar. 24, 2025. [Online]. Available: <https://www.mdpi.com/2227-7390/13/6/893>
- [25] "Improved SSD network for accurate detection of optic disc and fovea and application in excyclotropia screening | Request PDF." Accessed: Mar. 24, 2025. [Online]. Available: https://www.researchgate.net/publication/347770835_Improved_SSD_network_for_accurate_detection_of_optic_disc_and_fovea_and_application_in_excyclotropia_screening
- [26] H. Abdelali, H. Derrouz, Y. Zennayi, O. haj thami Rachid, and F. Bourzeix, "Multiple Hypothesis Detection and Tracking Using Deep Learning for Video Traffic Surveillance," *IEEE Access*, vol. PP, pp. 1–1, Dec. 2021, doi: 10.1109/ACCESS.2021.3133529.
- [27] "Acoustic Deterrents 2017.pdf."
- [28] R. L. Putland and A. F. Mensinger, "Acoustic deterrents to manage fish populations," *Rev. Fish Biol. Fish.*, vol. 29, no. 4, pp. 789–807, Dec. 2019, doi: 10.1007/s11160-019-09583-x.
- [29] "Götz and Janik - 2015 - Target-specific acoustic predator deterrence in th.pdf."
- [30] "MultiSound Acoustic Bird and Pigeon Deterrent, Turtle Doves, Starlings, Seagulls, Crows, Woodpeckers, Hoopoe - Bird and Animal Deterrent Sound (BS-20L - Agriculture 22) : Amazon.de: Garden." Accessed: Mar. 24, 2025. [Online]. Available: <https://www.amazon.de/-/en/MultiSound-Acoustic-Deterrent-Starlings-Woodpeckers/dp/B08BN4JCF1?th=1>
- [31] "Seiche | Acoustic Deterrent Device | Protecting Marine Mammals," Seiche. Accessed: Mar. 24, 2025. [Online]. Available: <https://www.seiche.com/underwater-acoustic-products/specialist-systems/acoustic-deterrent-device/>
- [32] "Götz and Janik - 2013 - Acoustic deterrent devices to prevent pinniped dep.pdf."
- [33] "Singh et al. - 2024 - Integrating Ultrasonic Sound Technology for Wild B.pdf." Accessed: Dec. 03, 2024. [Online]. Available: <https://www.wecmelive.com/open-access/integrating-ultrasonic-sound-technology-for-wild-boar-deterrence-in-agriculture-a-comprehensive-research-analysis.pdf>
- [34] "Overview_of_a_Personal_Network_Prototype.pdf."
- [35] "Sobin - 2020 - A Survey on Architecture, Protocols and Challenges.pdf."
- [36] "Khalifeh et al. - 2021 - Wireless Sensor Networks for Smart Cities Network.pdf."
- [37] A. Sendin, P. Losada-Sanisidro, J. García-Rodríguez, P. González-Méndez, and I. Berganza, "Broadband Over Power Line Communication Prototype Development for Next Generation Smart Meters: Validation in Access Electric Power Distribution Networks," *IEEE Access*, vol. 12, pp. 30191–30208, 2024, doi: 10.1109/ACCESS.2024.3367987.
- [38] "distributed-network-safety_research.pdf."
- [39] F. Windisch, K. Abedi, G. T. Nguyen, and T. Strufe, "Demo: Towards Rapid Prototyping Network-Slicing Solutions in Software-Defined Networks," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, Seoul, Korea, Republic of: IEEE, May 2024, pp. 1–3. doi: 10.1109/NOMS59830.2024.10575679.
- [40] "What Is Clean Code? A Guide to Principles and Best Practices." Accessed: Mar. 20, 2025. [Online]. Available: <https://blog.codacy.com/what-is-clean-code>

- [41] "Part 2: Folder Structure - Building a Solid Foundation," DEV Community. Accessed: Mar. 20, 2025. [Online]. Available: <https://dev.to/sathishskdev/part-2-folder-structure-building-a-solid-foundation-omh>
- [42] J. Itt, "Interpreted or Compiled?," Medium. Accessed: Mar. 20, 2025. [Online]. Available: <https://medium.com/@jonas.itt/interpreted-or-compiled-33c8e9bbc010>
- [43] "Difference between Compiled and Interpreted Language," GeeksforGeeks. Accessed: Mar. 20, 2025. [Online]. Available: <https://www.geeksforgeeks.org/difference-between-compiled-and-interpreted-language/>
- [44] "(23) Best Practices for Folder Structure in Project Management | LinkedIn." Accessed: Mar. 20, 2025. [Online]. Available: <https://www.linkedin.com/pulse/best-practices-folder-structure-project-management-gdcpm-solutions-hjvwf/>
- [45] M. Chiamonte, "Developing modular software: Top strategies and best practices," vFunction. Accessed: Mar. 21, 2025. [Online]. Available: <https://vfunction.com/blog/modular-software/>
- [46] P. Karsh, "GRASP Principles: Object-Oriented Design Patterns," Medium. Accessed: Mar. 25, 2025. [Online]. Available: <https://patrickkarsh.medium.com/object-oriented-design-with-grasp-principles-8049fa63e52>
- [47] "jetson-orin-datasheet-nano-developer-kit-3575392-r2.pdf." Accessed: Mar. 21, 2025. [Online]. Available: <https://nvdam.widen.net/s/zkfjmtsd2/jetson-orin-datasheet-nano-developer-kit-3575392-r2>
- [48] "Jetson Orin Nano Developer Kit Getting Started Guide," NVIDIA Developer. Accessed: Mar. 21, 2025. [Online]. Available: <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>
- [49] "10PCS Arduino Ultrasonic Module HC-SR04 Distance Sensor Measuring Transducer," eBay. Accessed: Apr. 20, 2025. [Online]. Available: <https://www.ebay.ca/itm/185360835316>
- [50] engineer.ahsin, "Manuel d'utilisation du capteur de mouvement infrarouge passif JOY-it SEN-HC-SR501," Manuals+. Accessed: Apr. 20, 2025. [Online]. Available: <https://manuals.plus/fr/joy-it/sen-hc-sr501-passive-infrared-motion-sensor-manual>
- [51] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," May 09, 2016, *arXiv*: arXiv:1506.02640. doi: 10.48550/arXiv.1506.02640.
- [52] S. S. A. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, "A Survey of Modern Deep Learning based Object Detection Models," May 12, 2021, *arXiv*: arXiv:2104.11892. doi: 10.48550/arXiv.2104.11892.
- [53] J. Terven and D. Cordova-Esparza, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," Jan. 07, 2024, *arXiv*: arXiv:2304.00501. doi: 10.48550/arXiv.2304.00501.
- [54] M. Dua and A. K. Jain, *Handbook of Research on Machine Learning Techniques for Pattern Recognition and Information Security*. IGI Global, 1AD. Accessed: Dec. 03, 2024. [Online]. Available: <https://www.igi-global.com/book/handbook-research-machine-learning-techniques/www.igi-global.com/book/handbook-research-machine-learning-techniques/241185>
- [55] T. Wi, M. Yang, S. Park, and J. Jeong, "D2-SPDM: Faster R-CNN-Based Defect Detection and Surface Pixel Defect Mapping with Label Enhancement in Steel Manufacturing Processes," *Appl. Sci.*, vol. 14, no. 21, Art. no. 21, Jan. 2024, doi: 10.3390/app14219836.
- [56] I. Rasmussen, S. Kvalsvik, P.-A. Andersen, T. N. Aune, and D. Hagen, "Development of a Novel Object Detection System Based on Synthetic Data Generated from Unreal Game Engine," *Appl. Sci.*, vol. 12, no. 17, Art. no. 17, Jan. 2022, doi: 10.3390/app12178534.
- [57] "YOLO-Tomato: A Robust Algorithm for Tomato Detection Based on YOLOv3." Accessed: Mar. 25, 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/20/7/2145>
- [58] "YOLOv8 vs Mask R-CNN: In-depth Analysis and Comparison," Labelvisor. Accessed: Dec. 03, 2024. [Online]. Available: <https://www.labelvisor.com/yolov8-vs-mask-r-cnn-in-depth-analysis-and-comparison/>

- [59] "YOLOv8 vs Faster R-CNN: A Comparative Analysis." Accessed: Nov. 27, 2024. [Online]. Available: https://keylabs.ai/blog/yolov8-vs-faster-r-cnn-a-comparative-analysis/?utm_source=chatgpt.com
- [60] M. Cascella *et al.*, "Employing the Artificial Intelligence Object Detection Tool YOLOv8 for Real-Time Pain Detection: A Feasibility Study," *J. Pain Res.*, vol. Volume 17, pp. 3681–3696, Nov. 2024, doi: 10.2147/JPR.S491574.
- [61] N. Dwivedi, "YOLOv11: The Next Leap in Real-Time Object Detection," Analytics Vidhya. Accessed: Nov. 29, 2024. [Online]. Available: <https://www.analyticsvidhya.com/blog/2024/10/yolov11/>
- [62] Ultralytics, "YOLO11  NEW." Accessed: Mar. 21, 2025. [Online]. Available: <https://docs.ultralytics.com/models/yolo11>
- [63] M. L. Ali and Z. Zhang, "The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection," *Computers*, vol. 13, no. 12, Art. no. 12, Dec. 2024, doi: 10.3390/computers13120336.
- [64] S. Srivastava, A. V. Divekar, C. Anilkumar, I. Naik, V. Kulkarni, and V. Pattabiraman, "Comparative analysis of deep learning image detection algorithms," *J. Big Data*, vol. 8, no. 1, p. 66, May 2021, doi: 10.1186/s40537-021-00434-w.
- [65] "SMS-Net: Bridging the Gap Between High Accuracy and Low Computational Cost in Pose Estimation." Accessed: Mar. 25, 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/14/22/10143>
- [66] K. Utkarsh, "Understanding Transfer Learning." Accessed: Mar. 25, 2025. [Online]. Available: <https://www.csharp.com/article/understanding-transfer-learning/>
- [67] S. O. Ajakwe, V. U. Ihekoronye, D.-S. Kim, and J.-M. Lee, "ALIEN: Assisted Learning Invasive Encroachment Neutralization for Secured Drone Transportation System," *Sensors*, vol. 23, no. 3, Art. no. 3, Jan. 2023, doi: 10.3390/s23031233.
- [68] G. Jocher, *YOLOv5 by Ultralytics*. (May 2020). Python. doi: 10.5281/zenodo.3908559.
- [69] G. Jocher, J. Qiu, and A. Chaurasia, *Ultralytics YOLO*. (Jan. 2023). Python. Accessed: Mar. 21, 2025. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [70] "Roboflow: Computer vision tools for developers and enterprises." Accessed: Mar. 21, 2025. [Online]. Available: <https://roboflow.com/>
- [71] John, *dog*. 2015. Accessed: Mar. 24, 2025. [Photo]. Available: <https://www.flickr.com/photos/luxuriel/20124303512/>
- [72] "Animal Buffalo Mammal - Free photo on Pixabay." Accessed: Mar. 24, 2025. [Online]. Available: <https://pixabay.com/photos/animal-buffalo-mammal-species-6786280/>
- [73] "GitHub - wkentaro/labelme: Image Polygonal Annotation with Python (polygon, rectangle, circle, line, point and image-level flag annotation)." Accessed: Mar. 21, 2025. [Online]. Available: <https://github.com/wkentaro/labelme>
- [74] "-Bilder: Stock-Fotos & -Videos,," Adobe Stock. Accessed: Mar. 24, 2025. [Online]. Available: <https://stock.adobe.com/de/images>
- [75] S. Rath, "YOLOv8 : Comprehensive Guide to State of the Art Object Detection." Accessed: Mar. 21, 2025. [Online]. Available: <https://learnopencv.com/ultralytics-yolov8/>
- [76] "Get Started with Training a YOLOv8 Object Detection Model." Accessed: Mar. 21, 2025. [Online]. Available: <https://www.datature.io/blog/get-started-with-training-a-yolov8-object-detection-model>
- [77] "ImageMagick – Contrast Limited Adaptive Histogram Equalization," ImageMagick. Accessed: Apr. 21, 2025. [Online]. Available: <https://imagemagick.org/>
- [78] Kukil, "Object Detection using YOLOv5 OpenCV DNN in C++ and Python." Accessed: Mar. 22, 2025. [Online]. Available: <https://learnopencv.com/object-detection-using-yolov5-and-opencv-dnn-in-c-and-python/>
- [79] "Ultralytics · GitHub." Accessed: Mar. 22, 2025. [Online]. Available: <https://github.com/ultralytics>
- [80] "PyTorch documentation — PyTorch 2.6 documentation." Accessed: Mar. 22, 2025. [Online]. Available: <https://pytorch.org/docs/stable/index.html>

- [81] "NumPy documentation — NumPy v2.2 Manual." Accessed: Mar. 22, 2025. [Online]. Available: <https://numpy.org/doc/stable/>
- [82] Ultralytics, "Home." Accessed: Mar. 22, 2025. [Online]. Available: <https://docs.ultralytics.com/>
- [83] "ONNX 1.18.0 documentation." Accessed: Mar. 22, 2025. [Online]. Available: https://onnx.ai/onnx/?utm_source=chatgpt.com
- [84] "TensorRT Documentation — NVIDIA TensorRT Documentation." Accessed: Mar. 22, 2025. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html>
- [85] "Configuring the Jetson Expansion Headers — Jetson Linux Developer Guide documentation." Accessed: Apr. 18, 2025. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r35.4.1/DeveloperGuide/text/HR/ConfiguringTheJetsonExpansionHeaders.html>
- [86] "Problems with Jetpack 6 and GPIO - Jetson & Embedded Systems / Jetson Orin Nano," NVIDIA Developer Forums. Accessed: Apr. 18, 2025. [Online]. Available: <https://forums.developer.nvidia.com/t/problems-with-jetpack-6-and-gpio/326340>
- [87] "A000067-datasheet.pdf." Accessed: Apr. 18, 2025. [Online]. Available: <https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf>
- [88] E. Casas, L. Ramos, E. Bendek, and F. Rivas-Echeverria, "YOLOv5 vs. YOLOv8: Performance Benchmarking in Wildfire and Smoke Detection Scenarios," *J. Image Graph.*, vol. 12, no. 2, pp. 127–136, 2024, doi: 10.18178/joig.12.2.127-136.
- [89] Ultralytics, "YOLOv8 vs YOLOv5: A Detailed Comparison." Accessed: Mar. 23, 2025. [Online]. Available: <https://docs.ultralytics.com/compare/yolov8-vs-yolov5>
- [90] "3.000+ kostenlose Water Buffalo und Water-Bilder - Pixabay." Accessed: Mar. 24, 2025. [Online]. Available: <https://pixabay.com/de/images/search/water%20buffalo/>
- [91] "Flickr Search — 'Nilgais,'" Flickr. Accessed: Mar. 24, 2025. [Online]. Available: <https://www.flickr.com/search/?text=Nilgais>
- [92] Z. Wang, Q. She, and T. E. Ward, "Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy," Dec. 29, 2020, *arXiv: arXiv:1906.01529*. doi: 10.48550/arXiv.1906.01529.

Appendix

```

1  import cv2
2  import torch
3  import numpy as np
4  from models.common import DetectMultiBackend
5  from utils.general import non_max_suppression, scale_boxes
6  from utils.torch_utils import select_device
7  from utils.plots import Annotator
8
9  def preprocess_frame(frame):
10     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
11     filtered = cv2.bilateralFilter(frame, 5, 50, 50)
12     lab = cv2.cvtColor(filtered, cv2.COLOR_BGR2LAB)
13     l, a, b = cv2.split(lab)
14     clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(6, 6))
15     l = clahe.apply(l)
16     lab = cv2.merge((l, a, b))
17     enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
18
19
20     sharpen_kernel = np.array([[0, -0.5, 0],
21                                [-0.5, 3, -0.5],
22                                [0, -0.5, 0]])
23
24     sharpened = cv2.filter2D(enhanced, -1, sharpen_kernel)
25     gamma = 1.1
26     inv_gamma = 1.0 / gamma
27     table = np.array([(i / 255.0) ** inv_gamma * 255 for i in np.arange(0, 256)]).astype("uint8")
28     corrected = cv2.LUT(sharpened, table)
29
30     return corrected
31
32 device = select_device('cuda:0' if torch.cuda.is_available() else 'cpu')
33 weights_path = "C:/yolov5/runs/train/YOLO5_1M/weights/best.pt"
34 model = DetectMultiBackend(weights_path, device=device)
35 cap = cv2.VideoCapture(0)
36 if not cap.isOpened():
37     print("Error: Camera not detected.")
38     exit()
39
40 while True:
41     ret, frame = cap.read()
42     if not ret:
43         print("Error: Could not read frame.")
44         break
45     frame_enhanced = preprocess_frame(frame)
46     frame_resized = cv2.resize(frame_enhanced, (800, 800))
47     img = torch.from_numpy(frame_resized).to(device)
48     img = img.permute(2, 0, 1).float() / 255.0
49     img = img.unsqueeze(0)
50     pred = model(img)
51     pred = non_max_suppression(pred, iou_thres=0.5)
52
53     annotator = Annotator(frame_resized, line_width=2)
54     for det in pred:
55         if det is not None and len(det):
56             det[:, :4] = scale_boxes(img.shape[2:], det[:, :4], frame_resized.shape).round()
57             for *xyxy, conf, cls in reversed(det):
58                 if conf >= 0.80:
59                     detected = True
60                     label = f"{model.names[int(cls)]} {conf:.2f}"
61                     annotator.box_label(xyxy, label)
62
63     cv2.imshow("YOLOv5 Detection", annotator.result())
64     if cv2.waitKey(1) & 0xFF == ord('q'):
65         break
66 cap.release()
67 cv2.destroyAllWindows()

```

Figure 73. Model I Base Code Variant. Available on <https://github.com/hbo-bomb/Project-Nilbye.git>.


```

1  import cv2
2  import torch
3  from ultralytics import YOLO
4  import numpy as np
5
6  def preprocess_frame(frame):
7      gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
8      filtered = cv2.bilateralFilter(frame, 5, 50, 50)
9      lab = cv2.cvtColor(filtered, cv2.COLOR_BGR2LAB)
10     l, a, b = cv2.split(lab)
11     clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(6, 6))
12     l = clahe.apply(l)
13     lab = cv2.merge((l, a, b))
14     enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
15
16     sharpen_kernel = np.array([[0, -0.5, 0],
17                                [-0.5, 3, -0.5],
18                                [0, -0.5, 0]])
19
20     sharpened = cv2.filter2D(enhanced, -1, sharpen_kernel)
21     gamma = 1.1
22     inv_gamma = 1.0 / gamma
23     table = np.array([(i / 255.0) ** inv_gamma * 255 for i in np.arange(0, 256)]).astype("uint8")
24     corrected = cv2.LUT(sharpened, table)
25
26     return corrected
27
28 model_path = "C:/yolov5/runs/train/YOLO8_1M/weights/best.pt"
29 model = YOLO(model_path)
30 cap = cv2.VideoCapture(0)
31
32 if not cap.isOpened():
33     print("Error: Camera not detected.")
34     exit()
35
36 while True:
37     ret, frame = cap.read()
38     if not ret:
39         print("Error: Could not read frame.")
40         break
41
42     frame_enhanced = preprocess_frame(frame)
43
44     frame_resized = cv2.resize(frame_enhanced, (800, 800))
45     results = model(frame_resized)
46
47     height, width, _ = frame.shape
48     scale_x = width / 800
49     scale_y = height / 800
50
51     for result in results:
52         for box in result.boxes:
53             x1, y1, x2, y2 = map(int, box.xyxy[0])
54             confidence = float(box.conf[0])
55
56             if confidence >= 0.80:
57                 class_id = int(box.cls[0])
58                 label = f"{model.names[class_id]} {confidence:.2f}"
59                 x1 = int(x1 * scale_x)
60                 y1 = int(y1 * scale_y)
61                 x2 = int(x2 * scale_x)
62                 y2 = int(y2 * scale_y)
63                 cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
64                 cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
65
66     cv2.imshow("YOLOv8 Detection", frame)
67
68     if cv2.waitKey(1) & 0xFF == ord('q'):
69         break
70
71 cap.release()
72 cv2.destroyAllWindows()

```

Figure 74. Model II Base Code Variant. Available at <https://github.com/hbo-bomb/Project-Nilbye.git>.

```

1  int ledPin = 13;           // Pin connected to the LED
2  int buzzerPin1 = 7;       // Pin connected to the first passive buzzer (object detection)
3  int buzzerPin2 = 12;      // Pin connected to the second passive buzzer (PIR + ultrasonic sensor)
4  int pirPin = 8;          // Pin connected to the PIR sensor
5  int trigPin = 9;         // Pin connected to the ultrasonic sensor TRIG
6  int echoPin = 10;        // Pin connected to the ultrasonic sensor ECHO
7  long duration;
8  int distance;
9
10 bool sensorsActive = false; // Flag to activate/deactivate sensors
11
12 void setup() {
13   pinMode(ledPin, OUTPUT); // Set LED pin as output
14   pinMode(buzzerPin1, OUTPUT); // Set buzzer pin 1 as output (object detection)
15   pinMode(buzzerPin2, OUTPUT); // Set buzzer pin 2 as output (PIR + ultrasonic sensor)
16   pinMode(pirPin, INPUT); // Set PIR sensor pin as input
17   pinMode(trigPin, OUTPUT); // Set TRIG pin as output
18   pinMode(echoPin, INPUT); // Set ECHO pin as input
19
20   digitalWrite(ledPin, LOW); // Ensure the LED is off initially
21   noTone(buzzerPin1); // Ensure the first buzzer is off initially
22   noTone(buzzerPin2); // Ensure the second buzzer is off initially
23   Serial.begin(9600); // Start serial communication
24 }
25
26 void loop() {
27   // Check if Python has sent the signal to activate sensors
28   if (Serial.available()) {
29     char command = Serial.read(); // Read the incoming serial data
30     if (command == '1') {
31       sensorsActive = true; // Activate sensors
32     } else if (command == '0') {
33       sensorsActive = false; // Deactivate sensors
34       noTone(buzzerPin1); // Turn off buzzer 1
35       noTone(buzzerPin2); // Turn off buzzer 2
36     }
37   }
38
39   // If sensors are activated by Python, read sensor values
40   if (sensorsActive) {
41     // Measure the distance using the ultrasonic sensor
42     digitalWrite(trigPin, LOW);
43     delayMicroseconds(2);
44     digitalWrite(trigPin, HIGH);
45     delayMicroseconds(10);
46     digitalWrite(trigPin, LOW);
47     duration = pulseIn(echoPin, HIGH);
48     distance = duration * 0.034 / 2; // Convert duration to distance (cm)
49
50     // If PIR sensor detects motion or distance is less than threshold, trigger buzzer 2
51     if (digitalRead(pirPin) == HIGH || distance < 20) {
52       tone(buzzerPin2, 2000); // Trigger buzzer 2 with high frequency (2000 Hz) when motion or proximity detected
53     } else {
54       noTone(buzzerPin2); // Stop buzzer 2 if no motion or distance is above threshold
55     }
56   }
57
58   // Control LED and Buzzer 1 based on detection confidence from Python (independent of PIR/Ultrasonic sensors)
59   if (Serial.available()) {
60     char command = Serial.read(); // Read the incoming serial data
61     if (command == '1') {
62       digitalWrite(ledPin, HIGH); // Turn on the LED
63       tone(buzzerPin1, 1000); // Trigger buzzer 1 at a normal frequency (1000 Hz) when detection is found
64     } else if (command == '0') {
65       digitalWrite(ledPin, LOW); // Turn off the LED
66       noTone(buzzerPin1); // Turn off buzzer 1
67     }
68   }
69 }
--

```

Figure 75. Arduino Code. Available at <https://github.com/hbo-bomb/Project-Nilbye.git>.

```

1  import serial
2  import time
3  import cv2
4  import torch
5  import numpy as np
6  from models.common import DetectMultiBackend
7  from utils.general import non_max_suppression, scale_boxes
8  from utils.torch_utils import select_device
9  from utils.plots import Annotator
10
11  DAHUA_IP = "192.168.188.37"
12  USERNAME = "admin"
13  PASSWORD = "ganesh762"
14  rtsp_url = f"rtsp://{USERNAME}:{PASSWORD}@{DAHUA_IP}:554/cam/realmonitor?channel=1&subtype=1"
15  device = 'cuda' if torch.cuda.is_available() else 'cpu'
16  weights_path = "/home/lain/yolov5/runs_final/YOLO5_1M/weights/best.onnx"
17  model = DetectMultiBackend(weights_path, device=device)
18  ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
19  def preprocess_frame(frame):
20      gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
21      filtered = cv2.bilateralFilter(gray, 5, 50, 50)
22      lab = cv2.cvtColor(filtered, cv2.COLOR_GRAY2LAB)
23      l, a, b = cv2.split(lab)
24      clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(6, 6))
25      l = clahe.apply(l)
26      lab = cv2.merge((l, a, b))
27      enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
28
29      return enhanced
30  def control_led(state):
31      ser.write(b'1' if state else b'0')
32  def control_buzzer(state):
33      ser.write(b'1' if state else b'0')
34  def init_camera():
35      cap = cv2.VideoCapture(rtsp_url)
36      cap.set(cv2.CAP_PROP_BUFFERSIZE, 2)
37      cap.set(cv2.CAP_PROP_FPS, 5)
38      cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
39      cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
40      return cap
41  cap = init_camera()
42  if not cap.isOpened():
43      print("Error: Cannot open Main RTSP stream.")
44      exit()
45  else:
46      print("Main Stream is working with subtype=0!")
47
48  CONFIDENCE_THRESHOLD = 0.75
49  while True:
50      ret, frame = cap.read()
51      if not ret:
52          print("Error: Could not read frame.")
53          break
54      frame = preprocess_frame(frame)
55      frame_resized = cv2.resize(frame, (800, 800))
56      img = torch.from_numpy(frame_resized).to(device)
57      img = img.permute(2, 0, 1).float() / 255.0
58      img = img.unsqueeze(0)
59      pred = model(img)
60      pred = non_max_suppression(pred)
61      annotator = Annotator(frame, line_width=2)
62      detection_found = False
63
64      for det in pred:
65          if det is not None and len(det):
66              det[:, :4] = scale_boxes(img.shape[2:], det[:, :4], frame.shape).round()
67              for *xyxy, conf, cls in reversed(det):
68                  if conf >= CONFIDENCE_THRESHOLD:
69                      label = f"{model.names[int(cls)]} {conf:.2f}"
70                      annotator.box_label(xyxy, label)
71                      detection_found = True
72
73      if detection_found:
74          control_led(True)
75          control_buzzer(True)
76      else:
77          control_led(False)
78          control_buzzer(False)
79
80      cv2.imshow("Dahua RTSP Stream - Detections", annotator.result())
81      if cv2.waitKey(1) & 0xFF == ord('q'):
82          break
83  cap.release()
84  cv2.destroyAllWindows()

```

Figure 76. Model I Enhanced Code Variant. Available at <https://github.com/hbo-bomb/Project-Nilbye.git>.

```

1  import serial
2  import time
3  import cv2
4  import torch
5  import numpy as np
6  from ultralytics import YOLO
7
8  DAHUA_IP = "192.168.188.37"
9  USERNAME = "admin"
10 PASSWORD = "ganesh762"
11 rtsp_url = f"rtsp://{USERNAME}:{PASSWORD}@{DAHUA_IP}:554/cam/realmonitor?channel=1&subtype=1"
12 ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
13 def preprocess_frame(frame):
14     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15     filtered = cv2.bilateralFilter(gray, 5, 50, 50)
16     lab = cv2.cvtColor(filtered, cv2.COLOR_BGR2LAB)
17     l, a, b = cv2.split(lab)
18     clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(6, 6))
19     l = clahe.apply(l)
20     lab = cv2.merge((l, a, b))
21     enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
22     return enhanced
23 def control_led(state):
24     ser.write(b'1' if state else b'0')
25 def control_buzzer(state):
26     ser.write(b'1' if state else b'0')
27 def init_camera():
28     cap = cv2.VideoCapture(rtsp_url)
29     cap.set(cv2.CAP_PROP_BUFFERSIZE, 2)
30     cap.set(cv2.CAP_PROP_FPS, 5)
31     cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
32     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
33     return cap
34 model_path = "/home/lain/yolov5/runs_final/YOLO8_1M/weights/best.onnx"
35 model = YOLO(model_path)
36 cap = init_camera()
37 if not cap.isOpened():
38     print("Error: Cannot open Main RTSP stream.")
39     exit()
40 else:
41     print("Main Stream is working with subtype=1!")
42 CONFIDENCE_THRESHOLD = 0.60
43 while True:
44     ret, frame = cap.read()
45     if not ret:
46         print("Error: Could not read frame.")
47         break
48     frame = preprocess_frame(frame)
49     frame_resized = cv2.resize(frame, (800, 800))
50     results = model(frame_resized, verbose=False, device='cuda' if torch.cuda.is_available() else 'cpu')
51     detection_found = False
52     for result in results:
53         for box in result.boxes:
54             x1, y1, x2, y2 = map(int, box.xyxy[0])
55             confidence = float(box.conf[0])
56
57             if confidence >= CONFIDENCE_THRESHOLD:
58                 class_id = int(box.cls[0])
59                 label = f"{model.names[class_id]} {confidence:.2f}"
60
61                 cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
62                 cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
63
64             detection_found = True
65
66     if detection_found:
67         control_led(True)
68         control_buzzer(True)
69     else:
70         control_led(False)
71         control_buzzer(False)
72
73     cv2.imshow("YOLOv8 Detection - Dahua", frame)
74
75     if cv2.waitKey(1) & 0xFF == ord('q'):
76         break
77
78 cap.release()
79 cv2.destroyAllWindows()
80 ser.close()

```

Figure 77. Model II Enhanced Code Variant. Available at <https://github.com/hbo-bomb/Project-Nilbye.git>.

Affidavit

I hereby declare in my word of honour

- that I prepared this final thesis independently and without any outside help.
- that I used no sources or resources other than those indicated.
- that I marked any verbatim quotes and paraphrased text by other authors within the work where they appear.
- that I did not submit it elsewhere for examination purposes.

I am fully aware that a false declaration will have legal consequences.

Ich erkläre hiermit ehrenwörtlich,

- diese Master-/Bachelor-/Seminararbeit selbstständig und ohne fremde Hilfe angefertigt,
- keine anderen als die angegebenen Quellen und Hilfsmittel benutzt,
- die Übernahme wörtlicher und sinngemäßer Zitate aus der Literatur an den entsprechenden Stellen innerhalb der Arbeit gekennzeichnet,
- die Arbeit mit gleichem Inhalt bzw. in wesentlichen Teilen noch nicht anderweitig für Prüfungszwecke vorgelegt zu haben.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Schweinfurt, 22.04.2025

Ort, Datum



Unterschrift

The English text in this document only serves the purpose of providing information on the contents of the corresponding German text.
Only the German version of this affidavit is legally binding.